Rapport de stage — M1 AM

Exploration du paysage des Réseaux Booléens à Signe Pluripotents

---

Équipe BCM — Laboratoire TIMC, Grenoble

Quentin Boyer

Encadrants : Nicolas Glade, Laurent Trilling

Université Grenoble Alpes, Grenoble INP ENSIMAG

Mai-Juillet 2026

Résumé

Résumé. [À rédiger en dernier.]

Abstract. [English version of the abstract.]

Mots-clés : Systèmes biologiques, réseaux booléens à signe (SBN), pluripotence, robustesse, évolutivité, Answer Set Programming, calcul parallèle.

Table des matières

1 Introduction **4**

1.1 Motivations 4

1.2 Positionnement dans les travaux de l'équipe 6

1.3 Contributions de ce stage 6

2 Formalisme des Réseaux Booléens à Signe **7**

2.1 Structure et dynamique 7

2.2 Fonctions booléennes à seuil (SBF) 8

2.2.1 Calcul de la borne des poids 9

2.3 Quel paysage des réseaux explore-t-on ? 12

2.4 Statistiques, propriétés, métriques 16

2.5 Métagraphes dynamiques et structurels, règles de mutation 17

3 Méthodes et outils **20**

3.1 Correction 20

3.1.1 Le programme de Laurent 21

3.1.2 Arbre fixé 21

3.1.3 Arbre cherché par le solveur 24

3.1.4 Arbre étiqueté 25

3.1.5 Validation 26

3.2 Optimisation 26

3.3 Visualisation 30

3.4 Usage des grands modèles de langage en recherche 32

3.5 Exploration Randomisée : MCSBN 33

4 Résultats **34**

5 Conclusion, travaux inachevés et ouverture **38**

A Guide d'utilisation des programmes **41**

A.1 Organisation du dépôt et installation 41

A.2 Format des CSV 42

A.3 Programmes ASP d'énumération 43

A.4 JPPipeline : énumération parallèle et statistiques 44

A.5 MCSBN : exploration Monte-Carlo 45

A.6 Scripts annexes et programmes historiques 47

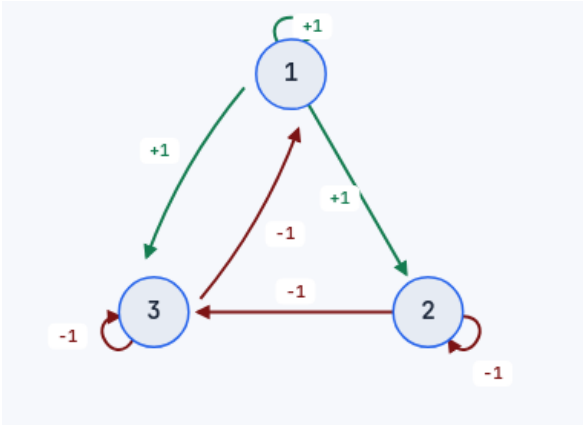
1 Introduction

1.1 Motivations

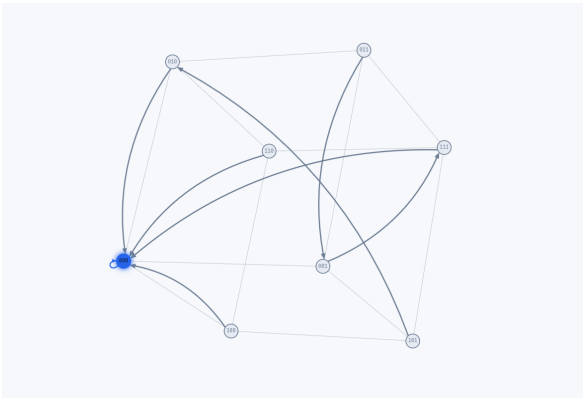
Un des intérêts de la modélisation des systèmes biologiques au sens large et à toute échelle (que ce soient des écosystèmes,des cellules, l'expression génétique, des relations symbiotiques, des neurones...) est qu'elle permet suffisamment de simplifications et d'abstractions pour pouvoir isoler (donc clarifier) et forger des concepts sur lesquels on peut facilement travailler (au sens mathématique) et dont on peut évaluer la pertinence (ce qui est un objectif en fin de compte, forger des concepts pertinents pour que ceux qui viennent après s'en emparent). L'étude in vivo (ou juste du vivant ?) ne peut généralement qu'entrapercevoir ces mêmes concepts qu'au milieu du fouillis des particularités biologiques, et encore, souvent au travers du prisme de la vie telle qu'on la connaît (prisme duquel on voudrait s'affranchir s'il est question d'origine du vivant, voire d'exobiologie), là où le modèle ouvre la voie à l'exhaustivité et à la généralisation (on a bien dit ouvre la voie, pas résout le problème). C'est donc dans ce cadre que nous nous plaçons : l'exploration des horizons d'un modèle (celui des Signed Boolean Network SBN) pour observer les contraintes qu'il impose et les lois que celles-ci font émerger (développer l'idée que contraindre permet ?). Il n'est donc pas question ici d'évaluer la validité biologique du modèle (même si nos réflexions à ce sujet guident nos choix) mais de bien modestement (j'ai deux mois, ce n'est pas énorme) en exploiter le potentiel conceptuel (et l'intérêt porté à cette exploitation est en quelque sorte une mesure de sa validité (i.e. utilité) conceptuelle).

La caractéristique du vivant qui sert de point de départ à ce travail est la pluripotence, c'est-à-dire la capacité d'un système unique à présenter différents comportements qui eux-mêmes correspondent à d'autres systèmes similaires mais potentiellement plus "simple", on dit que le système principal les "simule". Typiquement l'expression du code génétique (d'où proviendra souvent par analogie le vocabulaire que nous emploierons) dont la spécification des cellules est un bon exemple, des poils de chat court ou long selon la saison, la génération des pensées par nos neurones... [À fournir et préciser].

Le modèle exploré ici pour formaliser ces concepts est donc celui des Réseaux Booléen à Signe SBN, formalisé en Section 2 et ce qui nous y intéresse particulièrement, c'est l'interprétation des d nœuds du réseau comme une STRUCTURE (ou un programme, un code, un plan...) possédant différent états, et dont l'évolution déterministe génère une DYNAMIQUE (ou une exécution, une expression, une construction...) dans l'espace du graphe de transition de ces états (hypercube à d dimensions).



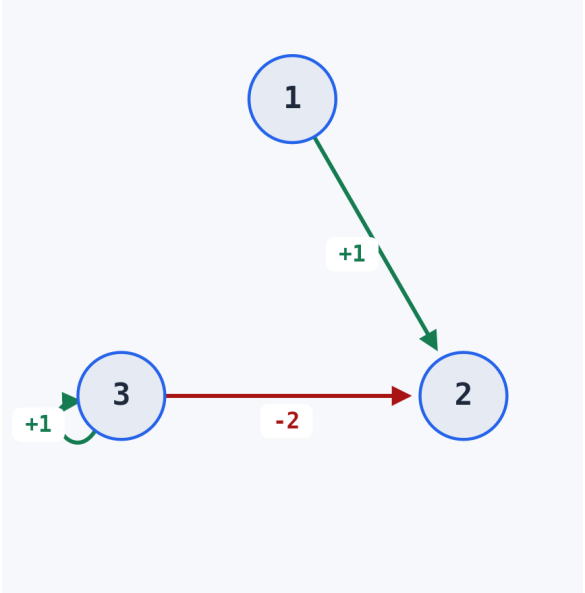
(a) Une structure quelconque



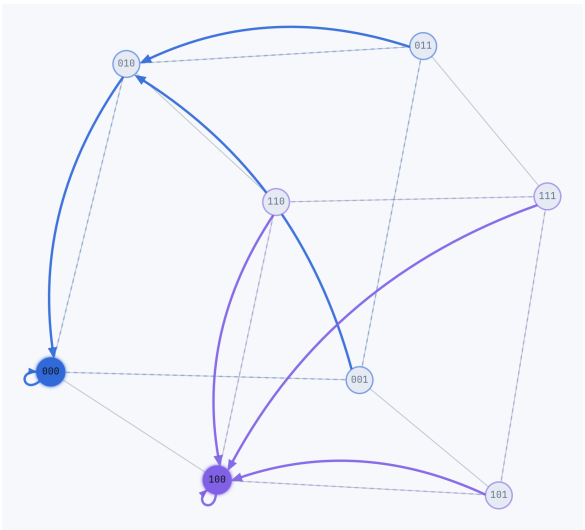
(b) Sa dynamique

FIGURE 1 – Un réseau et son graphe de transition en dimension 3.

Le concept de pluripotence est volontairement suffisamment général et flou pour pouvoir capter les différentes interprétations que le vivant peut en faire. Elle pourrait ainsi prendre sens dans le formalisme des SBN comme la présence de sous-dynamiques stables correspondant à des SBN sur des supports pouvant être encodé par une dimension inférieure. Mais ce n'est pas la seule manière, on pourrait aussi (et c'est plutôt ce qui est fait ici comme première approche plus simple) rechercher des sous-dynamiques disjointes sur des supports identifiés par un unique noeud (dit "de contrôle"). Le choix des mots est important et il nous appartient d'être clair sur l'aspect de la pluripotence que l'on met en évidence (Section 2.3).



(a) Une structure pluripotente. Le noeud 3 est un noeud de contrôle.



(b) Sa dynamique possède des hyper-faces disjointes (marquées par différentes couleurs).

FIGURE 2 – Un réseau pluripotent et son graphe de transition.

1.2 Positionnement dans les travaux de l'équipe

Le présent travail n'est pas isolé, il s'intègre dans une réflexion plus large menée de longue date par Nicolas Glade sur le vivant, et à laquelle ont participé (et participe) de nombreux chercheurs et étudiants (souvent dans l'équipe BCM du TIMC). Consulter leurs écrits s'est avéré très instructif (le lecteur y est largement invité) et aura permis en quelque sorte de "Se tenir sur les épaules des géants et voir plus loin, voir dans l'invisible, à travers l'espace et à travers le temps." - Jean-Claude Ameisen.

En voici une liste (non exhaustive, mais rassemblant les principaux).

- HDR de Nicolas Glade : la bible donnant le cadre général [1].

- Thèse de Rémi Segretain : réflexions générales sur les SBN et développement de la Java Parallel Pipeline (JPP) [2, 3] (Section 3.2)

- Rapports d'Ai-Ling Bonnet et de Rémy Defossez : approfondissement des travaux de Rémi [4, 5] (Sections 2.4 et 2.5).

- Rapport de Danwen Wu : Démonstration mathématique de l'équivalence entre l'existence de dynamiques séparées sur deux hyper-faces disjointes et de contraintes sur les poids du réseau [6] (Section 2.3).

- Programmes de Laurent Trilling : le point de départ de ce travail, l'énumération en Answer Set Programming (ASP) des réseaux pluripotents sous nœud de contrôle via la dynamique (Section 3.1).

Pour ce qui est des travaux encore en cours au moment de la rédaction, Corentin Lacaze s'occupe de la simulation d'agents dont les comportements sont intégralement générés par des SBN. L'objectif étant de les mettre en compétition pour observer des phénomènes évolutifs et comprendre comment cela se traduit en termes de SBN. La complémentarité entre l'aspect théorique du présent travail d'un côté et l'aspect expérimental des simulations de l'autre aura alimenté nos échanges qui nous auront permis l'un l'autre de nous guider mutuellement.

1.3 Contributions de ce stage

L'apport principal de ce travail est la recherche de la pluripotence dans les SBN. L'enjeu principal a donc été d'abord d'énumérer efficacement les SBN qui nous intéressent, la définition de ce qui nous intéresse étant souvent guidé par l'exigence d'efficacité (recherche de symétries). L'efficacité mémoire et temporelle est cruciale, les espaces de recherche explosent combinatoirement avec le nombre de noeuds du réseau et avec nos méthodes, il n'est en pratique pas raisonnable d'espérer l'exhaustivité à partir de 4 noeuds, où même d'avoir des données représentatives à partir de 5 noeuds (Tableau 2). Une bonne partie du temps de travail a donc été dédié à l'optimisation du code pour essayer de récolter le maximum d'information avec un minimum de calculs.

Dans un second temps l'objectif a été de créer des visualisations utiles et pratiques, à

la fois pour vérifier la justesse de nos programmes, mais aussi pour apprécier intuitivement l'espace des réseaux à travers différents angles et différentes métriques. Cela se matérialise par les trois scripts hmtl/javascript présents à la racine du git, permettant tous les trois de charger des fichiers CSV générés indépendamment et d'en afficher de manière concise le contenu.

- sbn_viz.html : structures et dynamiques détaillés.

- sbn_scatter.html : nuages des dynamiques placées selon différentes métriques.

- metagraphe.html : nuages des dynamiques placées les unes par rapport aux autres selon des règles de mutations.

2 Formalisme des Réseaux Booléens à Signe

2.1 Structure et dynamique

Le formalisme repris ici est celui du rapport d'Ai-Ling [4]. Un SBN de dimension d est un graphe orienté dont les d nœuds prennent des valeurs booléennes et dont chaque arc $i \to j$ porte un poids entier $w_{i,j}$, activateur s'il est positif et inhibiteur s'il est négatif. Un poids nul équivaut à l'absence d'arc, choix discuté en Section 2.2. C'est la matrice des poids $W = (w_{i,j}) \in \mathbb{Z}^{d\times d}$ qui constitue la STRUCTURE du réseau, telle que la représente la Figure 1a (c'est la transposée de celle du rapport d'Ai-Ling, qui note g_{ij} le poids de l'arc $j \to i$).

Un état du réseau est un vecteur $\mathbf{x} \in \{0,1\}^d$, où $x_i = 1$ quand le nœud i est actif, et la valeur suivante du nœud j s'obtient en sommant les poids des arcs qui lui viennent des nœuds actifs :

$$f_j(\mathbf{x}) = 1 \iff \sum_{i=1}^{d} w_{i,j}\, x_i > 0. \tag{1}$$

Autrement dit, le nœud s'allume quand les influences qu'il reçoit sont au total activatrices, et s'éteint sinon. La fonction f_j ne dépend donc que de la colonne j de W : il s'agit d'une fonction booléenne à seuil (SBF), objet de la section suivante. Tous les nœuds sont mis à jour en même temps, et le réseau définit ainsi une fonction de transition

$$F : \{0,1\}^d \to \{0,1\}^d, \qquad \mathbf{x} \mapsto \big(f_1(\mathbf{x}), \dots, f_d(\mathbf{x})\big), \tag{2}$$

qui constitue sa DYNAMIQUE. Elle se représente par son graphe de transition, où chaque état $\mathbf{x}$ pointe vers $F(\mathbf{x})$, les 2^d états étant placés aux sommets de l'hypercube de dimension d. Deux états voisins sur l'hypercube ne diffèrent que par la valeur d'un nœud (Figure 1b, où un état s'écrit $x_d \cdots x_1$, le nœud 1 à droite).

Comme chaque état a un unique successeur et que les états sont en nombre fini, toute trajectoire finit par entrer dans un attracteur (un point fixe ou un cycle), dont le bassin

d'attraction est formé des états qui y mènent. Le passage de la structure à la dynamique n'est pas injectif : il existe une infinité de matrices de poids pour un nombre fini de dynamiques (dont le Tableau 2 donne le compte). Une même dynamique est donc en général obtenue à partir de nombreuses structures, et c'est sur ce point que s'appuient les notions de robustesse et d'évolutivité de la Section 2.4.

2.2 Fonctions booléennes à seuil (SBF)

On appelle SBF de dimension d toute fonction f_w qui vaut 1 en $\mathbf{x}$ si et seulement si $\sum_i w_i x_i > 0$, pour un vecteur de poids $w \in \mathbb{Z}^d$ (une colonne de W), et l'on note $\mathcal{T}_d = \{f_w : w \in \mathbb{Z}^d\}$ leur ensemble. Toutes les fonctions booléennes ne sont pas des SBF, loin de là. Le seuil étant nul, une SBF vaut toujours 0 dans l'état $\mathbf{0}$, et certaines fonctions qui respectent cette condition échappent encore au seuil, comme $x_1 \wedge x_2$, qui demanderait à la fois $w_1 \leq 0$, $w_2 \leq 0$ et $w_1 + w_2 > 0$. Il ne reste ainsi que 6 SBF parmi les 16 fonctions de deux variables (Tableau 1).

TABLE 1 – Les 6 SBF distinctes en dimension 2. $f(\mathbf{x}) = 1$ ssi $\sum_i w_i x_i > 0$, avec $w_i \in \{-1, 0, 1\}$.

(w_1, w_2) repr.	$f(00)$	$f(01)$	$f(10)$	$f(11)$	Expression
$(-1, -1)$	0	0	0	0	0
$(1, -1)$	0	1	0	0	$x_1 \wedge \neg x_2$
$(-1, 1)$	0	0	1	0	$\neg x_1 \wedge x_2$
$(1, 0)$	0	1	0	1	x_1
$(0, 1)$	0	0	1	1	x_2
$(1, 1)$	0	1	1	1	$x_1 \vee x_2$

Seule la table de vérité d'une SBF est retenue ici, ce qui écarte ce travail, de deux façons, des classes d'équivalence de SBF de la thèse de Rémi [2] et de la HDR de Nicolas [1], et explique que le Tableau 2 ne retrouve pas leurs comptes. D'une part, ces classes regroupent les SBF qui ne diffèrent que par une permutation de leurs poids, c'est-à-dire les orbites de $\mathcal{T}_d$ sous l'action du groupe S_d des permutations des d entrées, dont l'ensemble est noté $\mathcal{T}_d/S_d$, mais elles sont comptées séparément ici (le tableau donne aussi le nombre d'orbites). D'autre part, Rémi fait d'un poids nul l'absence de l'arc qui le porte : une SBF dont un poids est nul est alors la projection d'une SBF de dimension inférieure, et forme une classe distincte de celle d'une SBF de même table de vérité sans poids nul. Aucune distinction n'est faite ici entre un arc absent et un arc présent mais sans effet sur la table de vérité. Cette distinction a pourtant un sens du point de vue de l'évolution, où une interaction sans effet forme un potentiel évolutif [2], et la prendre en compte serait un chemin à exploiter, qui n'a pas été suivi ici.

Le Tableau 2 rassemble ces comptes, ainsi que le nombre de dynamiques, celui des dynamiques pluripotentes au sens de la Section 2.3, et la borne des poids B_d, plus petit entier tel que toute SBF de dimension d se réalise avec des poids pris dans $[-B_d, B_d]$, dont le calcul est expliqué plus bas. Le nombre de dynamiques découle directement de celui des SBF, car chaque nœud a sa propre SBF et les colonnes de W se choisissent indépendamment : tout d-uplet de SBF est donc la dynamique d'un réseau. Il y a ainsi $|\mathcal{T}_d|^d$ dynamiques en dimension d, soit 32 768 en dimension 3 mais déjà $1{,}9 \cdot 10^{10}$ en dimension 4, et l'énumération exhaustive est hors de portée à partir de là.

TABLE 2 – Ordres de grandeur des SBF et des dynamiques selon la dimension d.

d	SBF $\lvert\mathcal{T}_d\rvert$	Orbites $\lvert\mathcal{T}_d/S_d\rvert$	Borne des poids B_d	Dynamiques $\lvert\mathcal{T}_d\rvert^d$	Décomposables en PSBN
1	2	2	1	2	1
2	6	4	1	36	5
3	32	10	2	32 768	658
4	370	36	3	$1{,}87 \cdot 10^{10}$	$1{,}19 \cdot 10^{7}$
5	11 292	224	5	$1{,}84 \cdot 10^{20}$	$4{,}76 \cdot 10^{14}$
6	1 066 044	3 024	9	$1{,}47 \cdot 10^{36}$	$1{,}22 \cdot 10^{27}$

2.2.1 Calcul de la borne des poids

Pour énumérer des réseaux, il faut borner leurs poids, et une borne inférieure à B_d ferait manquer des dynamiques, d'où l'importance de son calcul. Pour une SBF f, soient P et N les ensembles d'états où elle vaut 1 et 0, et $W(f) = \min\{\|w\|_\infty : w \in \mathbb{Z}^d,\ f_w = f\}$ le plus petit poids maximal qui la réalise. Les poids étant entiers, la condition stricte $\sum_i w_i x_i > 0$ équivaut à $\sum_i w_i x_i \geq 1$, ce qui permet d'écrire le calcul de $W(f)$ comme un programme linéaire en nombres entiers :

$$
\begin{aligned}
W(f) = \min_{w \in \mathbb{Z}^d,\ b \in \mathbb{Z}_{\geq 0}} \quad & b \\
\text{s.c.} \quad & \textstyle\sum_i w_i x_i \geq 1 \quad \forall\, \mathbf{x} \in P, \\
& \textstyle\sum_i w_i x_i \leq 0 \quad \forall\, \mathbf{x} \in N, \\
& -b \leq w_i \leq b \quad \forall\, i \in \{1, \ldots, d\},
\end{aligned}
\tag{3}
$$

dont les deux premières familles de contraintes imposent la table de vérité de f, et la dernière fait de b le plus grand des $|w_i|$. La borne B_d est alors le pire cas $\max_{f \in \mathcal{T}_d} W(f)$, qu'il faut donc calculer sur toutes les SBF.

Tester les 2^{2^d} fonctions booléennes n'est pas raisonnable à partir de $d = 5$, et aucune borne a priori (i.e. supérieure à la borne optimale) assez petite n'a été trouvée pour passer

par les poids. L'idée retenue est donc de parcourir $\mathcal{T}_d$ en largeur, deux SBF étant voisines quand elles ne diffèrent que sur un état. Pour une SBF f et un état $\mathbf{x} \neq \mathbf{0}$, la fonction g obtenue en changeant la valeur de f en $\mathbf{x}$ est une SBF si et seulement s'il existe des poids réels w tels que

$$\sum_i w_i y_i \geq 1 \quad \forall \mathbf{y} \in P(g), \qquad \sum_i w_i y_i \leq 0 \quad \forall \mathbf{y} \in N(g), \tag{4}$$

où $P(g)$ et $N(g)$ sont les ensembles d'états où g vaut 1 et 0. Ce sont les contraintes du programme précédent, sans borne ni objectif, et il suffit donc d'un simple test de faisabilité linéaire, résolu en réels, car un système de ce type qui admet une solution en admet une rationnelle, que la multiplication par le dénominateur commun rend entière. Encore faut-il que ce parcours ne manque aucune SBF, car une recherche par force brute sur des poids bornés peut, elle, manquer sans avertissement une SBF qui exigerait de plus gros poids. Le graphe dont les sommets sont les SBF de dimension d, deux SBF étant reliées quand elles ne diffèrent que sur un état, est en fait connexe, et un parcours en largeur parti de $f = 0$ les atteint donc toutes.

Démonstration. Les $2^d - 1$ hyperplans $H_{\mathbf{x}} = \{w \in \mathbb{R}^d : \sum_i w_i x_i = 0\}$, $\mathbf{x} \neq \mathbf{0}$, découpent l'espace des poids en régions ouvertes, les chambres, sur chacune desquelles le signe de chaque somme $\sum_i w_i x_i$ est constant. Chaque chambre définit donc une SBF, et on peut montrer que toute SBF est obtenue ainsi :

Soit f une SBF réalisée par w. Posons ε tel que

$$0 < \varepsilon < \min_{\mathbf{x} \in P(f)} \frac{\sum_i w_i x_i}{\sum_i x_i},$$

ce qui est possible car $P(f)$ est fini et ne contient pas $\mathbf{0}$ (si $P(f)$ est vide, tout $\varepsilon > 0$ convient), et posons w' tel que $w'_i = w_i - \varepsilon$ pour tout i. Soit $\mathbf{x} \in \{0,1\}^d$. Ou bien :

- $\mathbf{x} = \mathbf{0}$, auquel cas $\sum_i w'_i x_i = 0$;
- $\mathbf{x} \in N(f) \setminus \{\mathbf{0}\}$, auquel cas $\sum_i w'_i x_i = \underbrace{\sum_i w_i x_i}_{\leq 0} - \underbrace{\varepsilon \sum_i x_i}_{>0} < 0$;
- $\mathbf{x} \in P(f)$, auquel cas $\sum_i w'_i x_i = \sum_i w_i x_i - \varepsilon \sum_i x_i > 0$ par définition de ε.

Donc w' réalise f, et aucune somme n'est nulle pour $\mathbf{x} \neq \mathbf{0}$: w' n'est sur aucun hyperplan, il est dans une chambre.

Soient alors f et g deux SBF, réalisées par des vecteurs u et v pris dans des chambres. Le segment $[u, v]$, que l'on peut perturber légèrement pour qu'il ne traverse jamais deux hyperplans à la fois, traverse une suite de chambres, et chaque passage d'un hyperplan $H_{\mathbf{x}}$ ne change que le signe de la somme en $\mathbf{x}$. Deux chambres consécutives donnent donc deux SBF qui ne diffèrent qu'en $\mathbf{x}$, et cette suite relie f à g de voisine en voisine. □

La Figure 3 montre ce découpage en dimension 2, où les trois droites $w_1 = 0$, $w_2 = 0$

et $w_1 + w_2 = 0$ délimitent six chambres, qui sont les six SBF du Tableau 1. Tourner autour de l'origine fait passer d'une SBF à toutes les autres, en changeant un seul bit à chaque droite franchie. La figure illustre aussi le test (4). Depuis la chambre de x_1, changer sa valeur dans l'état 10 fait franchir la droite $w_2 = 0$ et donne $x_1 \vee x_2$, et la changer dans l'état 11 fait franchir la droite $w_1 + w_2 = 0$ et donne $x_1 \wedge \neg x_2$ (flèches de la figure). Changer sa valeur dans l'état 01 donnerait au contraire $x_1 \wedge x_2$, dont les contraintes $w_1 \leq 0$, $w_2 \leq 0$ et $w_1 + w_2 \geq 1$ sont incompatibles : la droite $w_1 = 0$ ne borde pas la chambre de x_1, et le test échoue. Cette figure reprend, à peu de chose près, la figure 12 de la thèse de Rémi [2], où l'espace des poids en dimension 2 est déjà découpé en classes colorées selon les permutations, et dont les illustrations aident à se représenter ce parcours, même si sa méthode d'énumération des SBF est différente (chapitres 3 et 4).

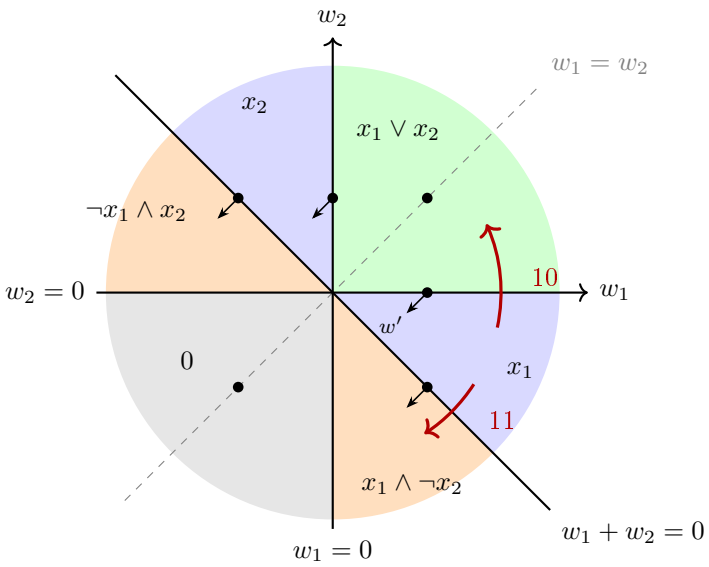


FIGURE 3 – Les six chambres de l'espace des poids en dimension 2, une par SBF. Les chambres de même couleur forment une orbite sous la permutation des entrées, qui échange w_1 et w_2 (symétrie par rapport à la droite $w_1 = w_2$). Les points sont les représentants du Tableau 1, et ceux qui tombent sur une droite entrent dans la chambre de leur SBF quand tous leurs poids diminuent de ε (petites flèches noires, de w à w' comme dans la preuve). Les flèches montrent les deux voisins de x_1, avec l'état dont la valeur change. D'après la figure 12 de la thèse de Rémi [2].

Deux SBF qui ne diffèrent que par une permutation de leurs entrées (en dimension 2, deux chambres de même couleur dans la Figure 3) ont par ailleurs la même borne W, car permuter les poids ne change pas leur maximum. Il suffit donc de résoudre le programme sur un représentant par orbite. Le parcours lui-même peut se restreindre aux orbites, car retourner un bit d'une SBF puis la permuter donne le même résultat que la permuter d'abord puis retourner le bit correspondant. Les voisins d'un seul représentant donnent donc déjà toutes les orbites voisines, et le calcul s'en trouve divisé par un facteur qui peut aller jusqu'à $d!$.

2.3 Quel paysage des réseaux explore-t-on ?

La pluripotence, telle que définie en introduction, peut prendre plusieurs formes dans le formalisme des SBN, et il s'agit surtout de présenter celles que ce travail explore, mais aussi celles qui paraissent intéressantes et qui l'ont moins été.

La première passe par les nœuds de contrôle. Un nœud c est dit de contrôle quand il garde toujours sa valeur, c'est-à-dire $f_c(\mathbf{x}) = x_c$ pour tout état $\mathbf{x}$. Cela rend invariantes les deux faces $x_c = 0$ et $x_c = 1$ de l'hypercube, et chacune contient alors une sous-dynamique de dimension $d-1$ que le réseau « simule » selon la valeur de c (Figure 2). Le rapport de Danwen [6] permet de lire cette propriété directement sur les poids : c est un nœud de contrôle si et seulement si

$$\sum_{i \neq c} \max(0, w_{i,c}) \leq 0 \qquad \text{et} \qquad w_{c,c} + \sum_{i \neq c} \min(0, w_{i,c}) > 0, \tag{5}$$

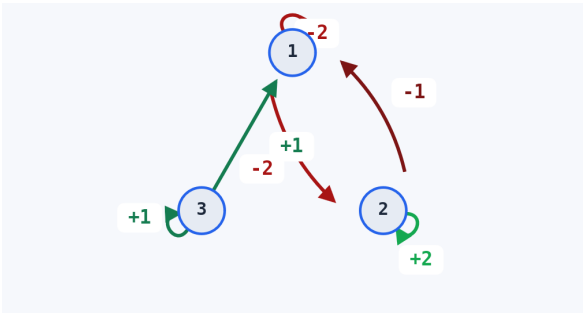
c'est-à-dire si aucun autre nœud ne peut l'allumer (ses arcs entrants sont inhibiteurs ou absents) ni l'éteindre (sa boucle l'emporte sur toutes les inhibitions réunies). C'est cette forme, avec les parties positives et négatives des poids, qu'implémentent exactement les programmes ASP (Section 3.1) à chaque coupe de l'arbre, en y ajoutant les poids venant des nœuds de contrôle déjà allumés. Elle est équivalente à celle du rapport de Danwen, qui écrit la seconde condition $w_{c,c} > \sum_{i \neq c} |w_{i,c}|$, les deux coïncidant puisque la première impose $w_{i,c} \leq 0$. Rien n'empêche chaque face d'avoir à son tour un nœud de contrôle parmi les nœuds restants, et ainsi de suite. Une dynamique se décompose donc selon un arbre, dont chaque nœud interne est une coupe selon un nœud de contrôle et chaque feuille un sous-cube invariant qui n'est plus coupé. Le vecteur de décomposition $\langle v_d, \ldots, v_0 \rangle$, où v_k est le nombre de feuilles de dimension k, va du vecteur trivial $\langle 1, 0, \ldots, 0 \rangle$, qui ne coupe rien, à $\langle 0, \ldots, 0, 2^d \rangle$, où tous les états sont des points fixes. Ces décompositions sont dites complètes, car leurs feuilles recouvrent tout l'hypercube, et ce sont elles qui sont énumérées par les programmes de la Section 3.1.

Ces sous-dynamiques ne sont pas toujours des SBN. Sur la face $x_c = 0$, le nœud c éteint ne contribue à rien : la sous-dynamique est celle du réseau restreint aux autres nœuds, donc celle d'un SBN. Mais sur la face $x_c = 1$, chaque nœud j reçoit toujours le poids $w_{c,j}$, qui joue le rôle d'un seuil. Il s'agit donc d'un réseau de fonctions à seuil quelconques (TBN, pour *Threshold Boolean Network*), qui n'est pas forcément un SBN. L'appellation est celle de la HDR de Nicolas [1], où une fonction booléenne à seuil (TBF) vaut 1 quand la somme pondérée de ses entrées dépasse strictement un seuil propre au nœud. La HDR montre aussi que toute TBF peut être simulée par une SBF qui reçoit d'un nœud toujours allumé un arc de poids opposé au seuil, et c'est le rôle que joue ici le nœud de contrôle. On appelle donc PTBN un réseau pluripotent dont les sous-dynamiques sont des TBN, et PSBN un réseau dont chaque sous-dynamique est celle d'un SBN de dimension inférieure.

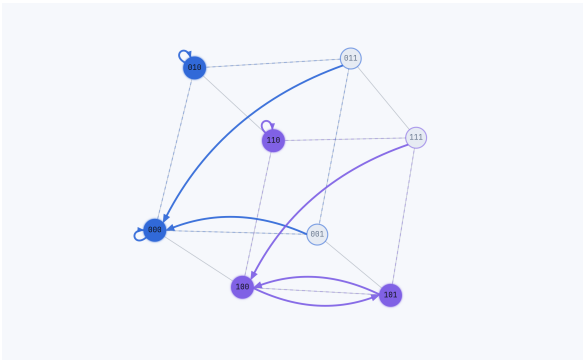
Cela demande, pour la face $x_c = 1$, l'existence de poids $w'_{i,j}$ tels que

$$\sum_{i \neq c} w_{i,j}\, x_i + w_{c,j} > 0 \iff \sum_{i \neq c} w'_{i,j}\, x_i > 0 \qquad \forall\, j \neq c,\ \forall \mathbf{x} \text{ tel que } x_c = 1, \tag{6}$$

et de même à chaque niveau de l'arbre, avec la somme des poids venant des nœuds de contrôle allumés. Comme un SBN ne peut pas sortir de l'état où tous ses nœuds sont éteints (aucune somme n'y est positive), il faut en particulier $w_{c,j} \leq 0$ pour tout $j \neq c$ [6]. C'est ce qui manque à la dynamique de la Figure 4, dont la face $x_3 = 1$ envoie l'état 100 sur 101. L'écart entre les deux notions est grand : en dimension 3, 2 977 dynamiques ont au moins un nœud de contrôle, mais 658 seulement se décomposent en SBN. Il est possible de compter ces dynamiques sans les énumérer. L'indépendance des colonnes de W permet en effet de se contenter de savoir, pour chaque SBF, si sa restriction à une face $x_c = 1$ est encore une SBF, puis de compter les d-uplets de SBF qui ont au moins un nœud de contrôle dont toutes les autres colonnes passent ce test. Il apparaît ainsi que la pluripotence devient vite rare (Tableau 2) : les dynamiques décomposables en SBN ne représentent que 2 % des dynamiques en dimension 3, 0,06 % en dimension 4 et 0,0003 % en dimension 5.



(a) La structure.

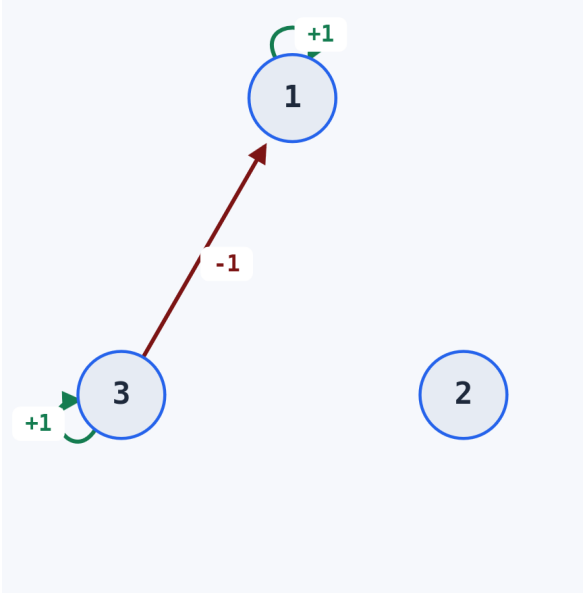


(b) Sa dynamique, coloriée par la décomposition.

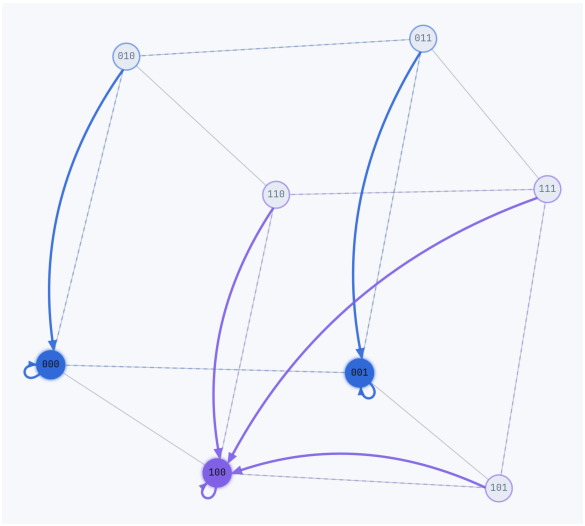
FIGURE 4 – Une dynamique décomposable en TBN mais pas en SBN. Le nœud 3 est un nœud de contrôle, mais sur la face $x_3 = 1$ l'état 100 va vers 101, ce qu'aucun SBN ne fait.

Une contrainte sur le domaine des poids eux-mêmes a aussi été tentée avec les PSBNI (I pour inhibiteurs), des PSBN dont les arcs entre nœuds distincts sont inhibiteurs ou absents, soit $w_{i,j} \leq 0$ pour $i \neq j$, les boucles restant libres. Cette piste n'a pas été maintenue jusqu'au bout : elle n'existe qu'avec un arbre de décomposition fixé, sans version à arbre étiqueté (Section 3.1), mais elle soulève un questionnement intéressant. La contrainte semble physiquement raisonnable, car on pourrait imaginer des contextes dans lesquels les arcs inhibiteurs seraient suffisamment distingués de ceux qui sont activateurs pour être discriminés positivement. Elle revient aussi à imposer à tous les nœuds la condition que les nœuds de contrôle vérifient déjà sur leurs arcs entrants. Les propriétés des poids

qu'exige la pluripotence y seraient donc beaucoup plus probables, et la contrainte pourrait apporter beaucoup en termes de pluripotence. C'est ce que suggère la dimension 3, où le nœud 3 est un nœud de contrôle (avec des sous-réseaux SBN) pour 36 des 216 dynamiques de ce régime, contre 225 des 32 768 dynamiques sans restriction.



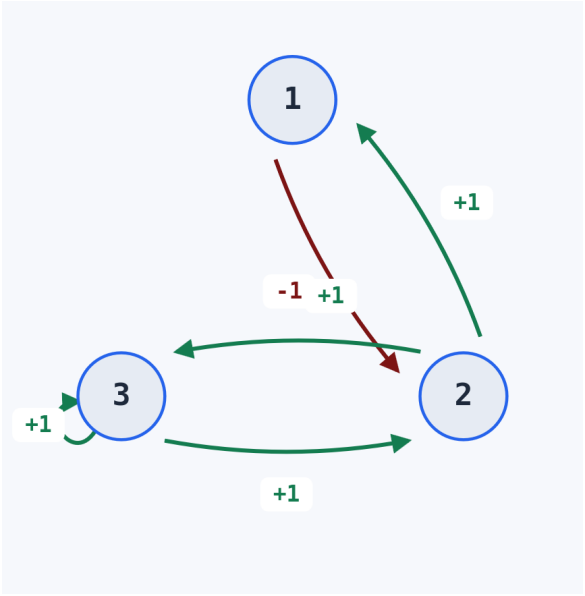
(a) La structure.



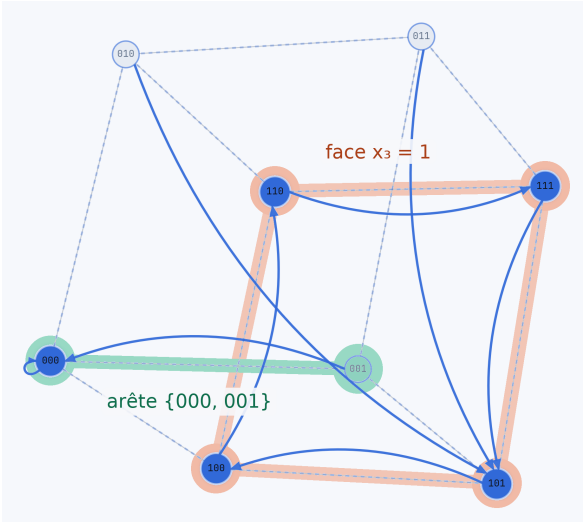
(b) Sa dynamique, coloriée par la décomposition.

FIGURE 5 – Un PSBNI, dont le seul arc entre nœuds distincts, de 3 vers 1, est inhibiteur, et dont le nœud 3 est un nœud de contrôle.

Une décomposition peut aussi n'être que partielle, quand certains sous-cubes de l'hypercube sont invariants sans que le reste se laisse découper. C'est le cas dans la Figure 6 : l'arête $\{000, 001\}$ et la face $x_3 = 1$ sont invariantes, mais les états 010 et 011 n'appartiennent à aucune des deux. Cette forme de pluripotence n'a pas été explorée ici (aucun des programmes n'énumère ces décompositions partielles).



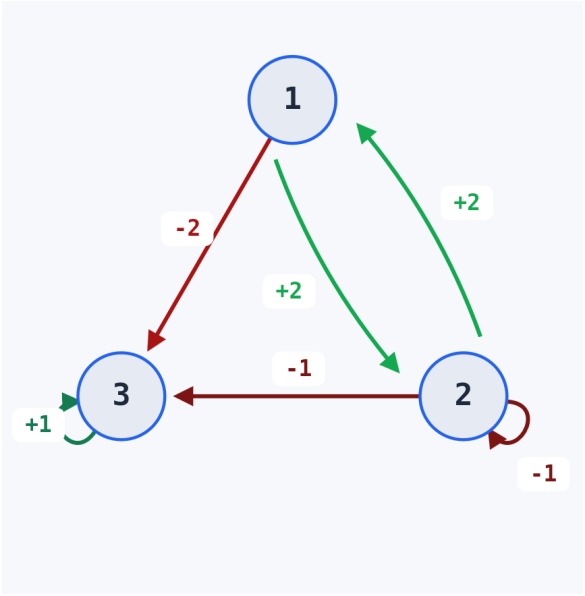
(a) La structure.



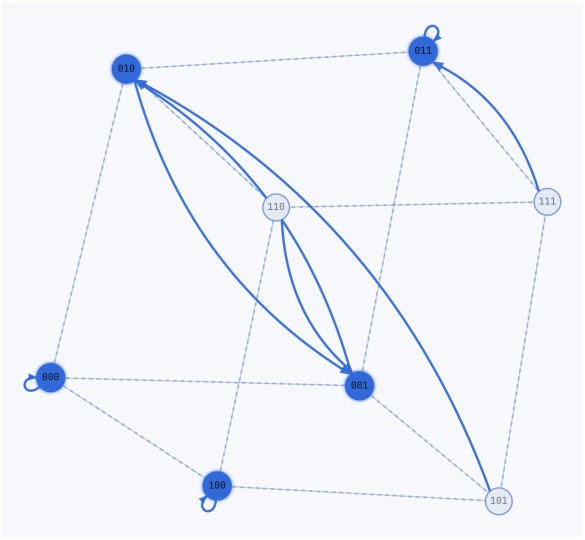
(b) Sa dynamique et ses deux sous-cubes invariants.

FIGURE 6 – Une dynamique sans décomposition complète, dont l'arête $\{000, 001\}$ et la face $x_3 = 1$ sont pourtant invariantes.

Les sous-dynamiques peuvent enfin être décrites par plusieurs coordonnées plutôt que par un seul bit de contrôle. Dans la dynamique de la Figure 7, les nœuds 1 et 2 évoluent seuls, comme un réseau de dimension 2 qui échange leurs valeurs, et le nœud 3 suit leur état : il garde sa valeur quand ils sont tous deux éteints, et s'éteint sinon. Il est possible d'y voir, comme dans l'architecture d'un processeur, une partie contrôle (les nœuds 1 et 2) qui pilote une partie opérative (le nœud 3). L'hypercube se partage alors en trois supports invariants, un par attracteur de la partie contrôle : $\{000, 100\}$, $\{011, 111\}$ et $\{001, 010, 101, 110\}$, ce dernier n'étant pas un sous-cube. Aucun arbre à un seul bit de contrôle ne capte cette organisation, et la dynamique n'a donc que le vecteur trivial. On parle alors d'offuscation, par analogie avec l'offuscation d'un code, qui en rend le comportement difficile à lire : les sous-dynamiques existent bien, mais comme la partie contrôle compte plusieurs nœuds, elles ne se distinguent plus selon les faces de l'hypercube. Cette piste des nœuds contrôlés, à peine esquissée pendant le stage, n'a pas été explorée.



(a) La structure, dont les nœuds 1 et 2 forment la partie contrôle et le nœud 3 la partie opérative.



(b) Sa dynamique, dont les supports invariants suivent l'attracteur de la partie contrôle.

FIGURE 7 – Un réseau organisé en partie contrôle et partie opérative.

2.4 Statistiques, propriétés, métriques

Donner des mesures fait parties des moyens que l'on a pour former des groupes, des relations d'équivalence, des relations d'ordre, bref pour se faire des intuitions sans avoir à éplucher l'intégralité de l'information contenue dans chaque réseau (ce qui est impossible, donc on fait une sorte de modèle dans le modèle). C'est pourquoi en plus d'énumérer les réseaux, nous calculons des métriques associés qui sont bien utiles pour visualiser les réseaux dans différents espaces. Le choix des statistiques calculées est motivé par les travaux de Rémi [2] et Ai-Ling [4], mais reviennent aussi beaucoup au papier de Wagner [7]. Un axe d'amélioration du présent travail serait les calculs d'autres statistiques, ce qui permettrait d'élargir l'horizon observable (dans les limites du passage à l'échelle nécessaire).

Ces statistiques reprennent le vocabulaire de Wagner [7]. Le génotype est la matrice des poids, prise parmi les matrices dont les poids sont bornés par B_d (Section 2.2), et le phénotype est la dynamique $P = (f_1, \ldots, f_d)$, dont on note $G(P)$ l'ensemble des génotypes qui la donnent. Une mutation remplace un seul poids par une autre valeur de $[-B_d, B_d]$, et chaque génotype a donc $K = 2B_d d^2$ mutants, une mutation étant dite neutre quand le mutant donne encore P (c'est la règle de mutation dont il est question en Section 2.5). Pour chaque dynamique P, les statistiques calculées sont les suivantes :

- le nombre de génotypes $|G(P)|$ (`GenotypeCount`) ;
- la robustesse moyenne (`Robustness_mean`), c'est-à-dire la moyenne sur $G(P)$ de la part $r(g)$ des K mutants de g qui sont neutres, et son écart-type (`Robustness_std`) ;

- l'évolutivité (`Evolvability`), c'est-à-dire le nombre de dynamiques différentes de P qu'atteint au moins un mutant d'au moins un génotype de $G(P)$;
- le nombre d'attracteurs (`NumAttractors`), qui est aussi celui des bassins d'attraction (Section 2.1) ;
- la moyenne sur les attracteurs du carré de leur longueur (`CycleLenMSQ`), un point fixe comptant pour 1.

Wagner estime les trois premières par échantillonnage, mais elles se calculent ici exactement, car f_j ne dépend que de la colonne j de W : $G(P)$ est donc le produit des ensembles de colonnes qui donnent chaque f_j, et une mutation ne touche qu'une colonne. Toutes ces statistiques s'obtiennent ainsi à partir de tables établies une fois par dimension.

La robustesse et l'évolutivité viennent du papier de Wagner, qui montre sur les séquences d'ARN que ces deux propriétés, opposées à l'échelle d'un génotype, vont de pair à l'échelle d'un phénotype. Ses mesures se transposent aux SBN, où un même phénotype est aussi obtenu à partir de nombreux génotypes, en nombres très inégaux. L'écart-type de la robustesse permet en plus de voir si les génotypes d'une même dynamique sont également robustes, mais les analogues de ces mesures à l'échelle d'un seul génotype n'ont pas été calculés. La corrélation positive entre robustesse et évolutivité des dynamiques que Wagner observe sur l'ARN se retrouve dans chacune des dimensions étudiées (Section 4).

Le nombre de génotypes d'une dynamique, que Wagner appelle la fréquence d'une structure, mesure sa rareté. La thèse de Rémi propose de rapporter la complexité d'un objet à sa rareté, un objet complexe étant coûteux à produire et donc rare [2], et l'applique aux SBF en mesurant la part de l'espace des poids qu'occupe chacune ; l'idée est ici étendue à la dynamique entière. Les deux statistiques des attracteurs résument la forme de la dynamique, et le carré des longueurs devait faire ressortir les cas extrêmes, un choix qui, avec le recul, n'est pas particulièrement pertinent.

Toutes ces statistiques ne dépendent que de la dynamique, pour une borne des poids donnée, car elles se calculent dans l'espace des matrices bornées par B_d. Une borne plus large les modifierait, comme le choix de l'intervalle des poids modifie la rareté des SBF chez Rémi [2]. Une statistique de la structure elle-même, comme la somme des poids, un temps calculée, demanderait de choisir une réalisation canonique de chaque dynamique, par exemple celle de poids minimaux. Cela n'a pas été fait, et les poids donnés pour une dynamique (dans les figures comme dans les CSV) n'en sont qu'une réalisation quelconque.

2.5 Métagraphes dynamiques et structurels, règles de mutation

Une approche proposée dans la thèse de Rémi (et mise en œuvre par Ai-Ling, à partir des distances formalisées par Rémy) a été de construire des méta-graphes dans lesquels deux réseaux sont reliés s'ils sont voisins en termes de distances (pouvant répondre à différentes définitions) et dont la position est calculée par un algorithme de type Force-

directed.

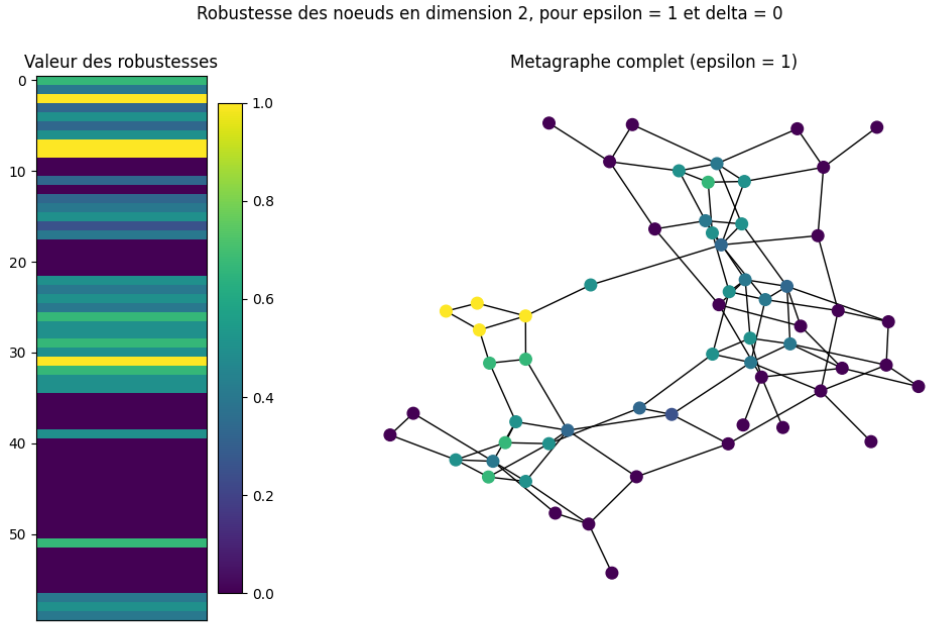


FIGURE 8 – Métagraphe structurel des 60 réseaux de dimension 2, deux réseaux étant reliés quand ils sont à distance structurelle 1, coloré selon la robustesse de chaque réseau, de 0 (violet) à 1 (jaune). Figure tirée du rapport d'Ai-Ling [4].

À noter qu'ici, les noeuds du graphe sont des réseaux entiers, là où ceux des graphes rencontrés jusqu'ici étaient les nœuds d'un réseau (dans les figures de structure) ou ses états (dans les graphes de transition), d'où le préfixe « méta », et dans les métagraphes construits plus loin, ce seront des dynamiques. Ces réseaux ne peuvent pas être identifiés à leurs dynamiques, car Ai-Ling distingue, comme Rémi (Section 2.2), deux réseaux de même dynamique dont les arcs fixés à zéro diffèrent : ils forment alors deux nœuds distincts, avec leurs propres voisins et leur propre robustesse (il y a ainsi 60 réseaux pour 36 dynamiques en dimension 2).

Il a été essayé de reproduire les mêmes méta-graphes en réutilisant les définitions de distances, en particulier la distance DA agrégeant les changements des cycles limites. Mais si le calcul exhaustif de ces distances reste faisable en dimension 3 (de l'ordre de 1e9 paires), le graphe obtenu y est trop dense pour être lisible, et ce calcul est difficilement concevable en dimension 4 (de l'ordre de 4e20 paires). Ce calcul n'a donc pas été poursuivi. Il est sans doute possible de trouver des manières de contourner cette difficulté qui n'est que technique, et de construire des méta-graphes similaires sémantiquement. En tout cas cette piste (qui existe) n'a pas été exploré plus avant ici.

Cependant l'idée de construire des méta-graphes Force-directed n'a pas été totalement mise de côté, nous nous sommes juste plus modestement limités à des méta-graphes structurels (ou les dynamiques sont liées si elles possèdent des structures voisines). Cela nous

permet notamment de faire un lien direct avec les simulations de Corentin Lacaze, car les règles de mutations qui dirige l'évolution des agents dans ce contexte sont des relations de voisinages entre structures. L'exploration contrainte par l'évolution des agents se traduit alors par un chemin (avec de potentiels embranchements, donc un arbre) dans le métagraphe, et cela permet de poser des questions du type "Est-ce que certaines propriétés des dynamiques favorise le passage des chemins", "Certaines zones sont-elles plus viables que d'autres ? C'est-à-dire est-ce que les chemins préfèrent certaines zones ?" "Pour arriver à un phénotype cible, est-ce que plusieurs chemins sont possibles, si oui en quoi sont-ils différents ?" .

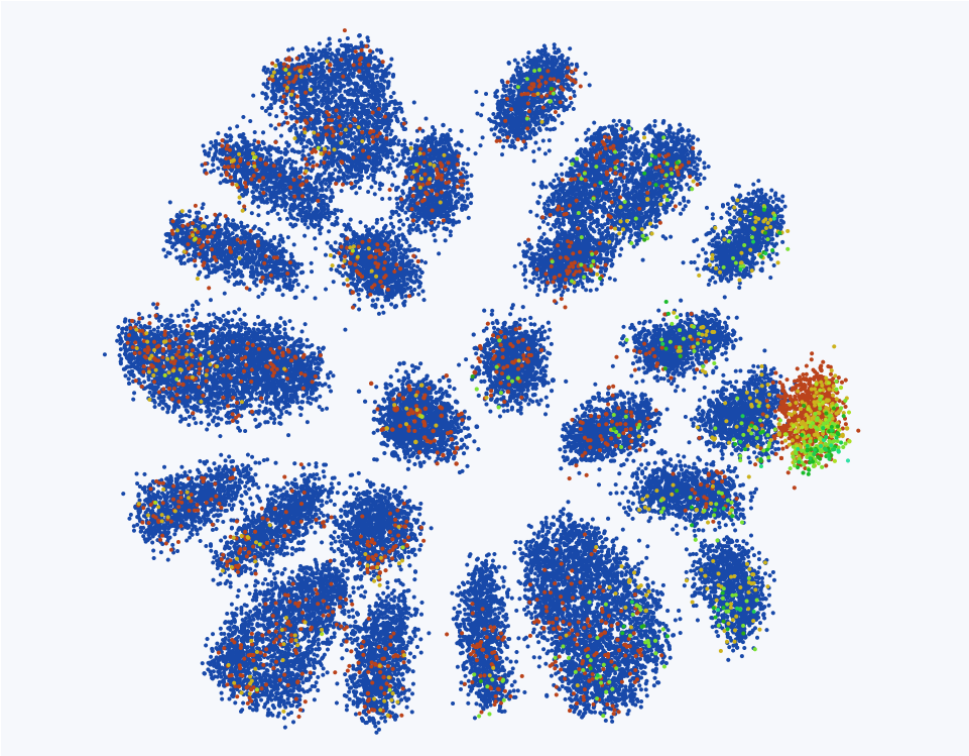


FIGURE 9 – Métagraphe structurel des 32 768 dynamiques de dimension 3, coloré selon leur décomposition la plus fine en PTBN.

La règle de mutation considérée en premier a donc été simplement celle d'un changement de poids quelconque (dans les limites de la borne inf des poids permettant d'exprimer toutes les dynamiques) sur un seul arc du réseau. Il serait intéressant de proposer d'autres règles de mutation pour observer les différences de résultats selon le prisme qu'elles constituent, et notamment des règles pouvant faire varier la dimension des réseaux : l'ajout ou la suppression de noeuds permettrait de prendre du recul par rapport à la dimension (qui reste un carcan solide dans cette étude).

Avec les notations de la Section 2.4, deux dynamiques distinctes P et P' sont donc reliées dans le métagraphe quand une mutation fait passer de l'une à l'autre, c'est-à-dire quand il existe $g \in G(P)$ et $g' \in G(P')$ qui ne diffèrent que par un poids. Une mutation ne touchant qu'une colonne de W, deux dynamiques reliées ne diffèrent que par la fonction

d'un seul nœud, et le degré d'une dynamique dans le métagraphe correspond alors à son évolutivité. Changer de règle de mutation demande de modifier ensemble les cinq endroits où elle est codée : `SBFStatTable.java` pour la robustesse et l'évolutivité calculées par JPP, `mcsbn.c` pour celles de MCSBN, `sbf.py` et `metagraph_layout.c` pour le placement du métagraphe, et `metagraphe.html` pour les arêtes affichées.

3 Méthodes et outils

Comme souvent dans l'écriture de code, les enjeux des phases de développement ont toujours été dans un premier temps d'arriver au résultat attendu (la correction) puis d'optimiser pour passer à l'échelle (la terminaison, idéalement rapide). Mais comme la correction dans notre cas nécessite surtout de bien savoir ce que l'on cherche, et que pour cela, il faut avoir des visualisations efficaces, les étapes de correction, d'optimisation et d'évaluation des rendus se sont succédé en cycles. Les sections suivantes ne reflètent donc pas nécessairement fidèlement la chronologie du développement.

3.1 Correction

L'objectif originel du stage était de prendre en main le language Answer Set Programming (ASP) pour parvenir à comprendre et améliorer les programmes existant (dossier `SBN_pluripotent_Laurent/` du dépôt).

L'ASP est un paradigme de programmation déclarative : il s'agit de décrire ce que l'on cherche par des règles et des contraintes logiques, plutôt que la manière de le trouver, et un solveur se charge ensuite d'énumérer les modèles (les *answer sets*) qui les satisfont. Ce cadre convient bien au problème, car les propriétés imposées aux réseaux (nœuds de contrôle, sous-réseaux SBN, forme de l'arbre) s'écrivent directement comme des contraintes. Le solveur permet aussi de projeter les solutions sur les seules fonctions de transition, et donc d'énumérer des dynamiques plutôt que toutes les matrices de poids qui les donnent. Le solveur utilisé est `clingo`, de Potassco[1], à travers son extension `clingcon` [8][2], qui ajoute des contraintes linéaires sur des variables entières et permet donc de manipuler les poids et leurs sommes. ApproxASP [9][3], un compteur approché de modèles, a aussi servi un temps à compter les génotypes d'une dynamique, avant que ce comptage ne devienne exact (Section 2.4).

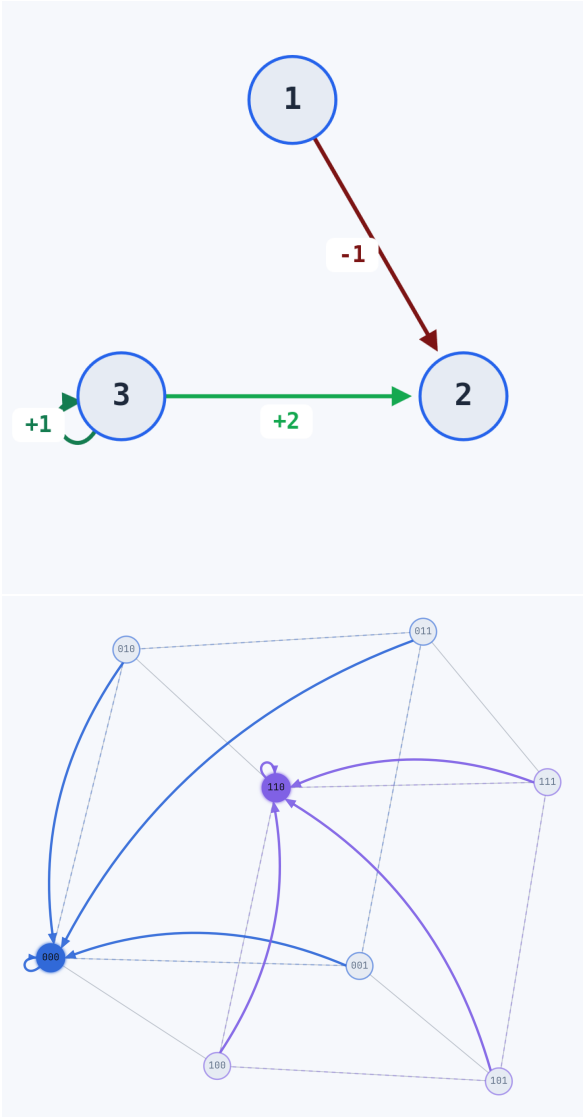
1. https://potassco.org
2. https://github.com/potassco/clingcon
3. https://github.com/meelgroup/ApproxASP

3.1.1 Le programme de Laurent

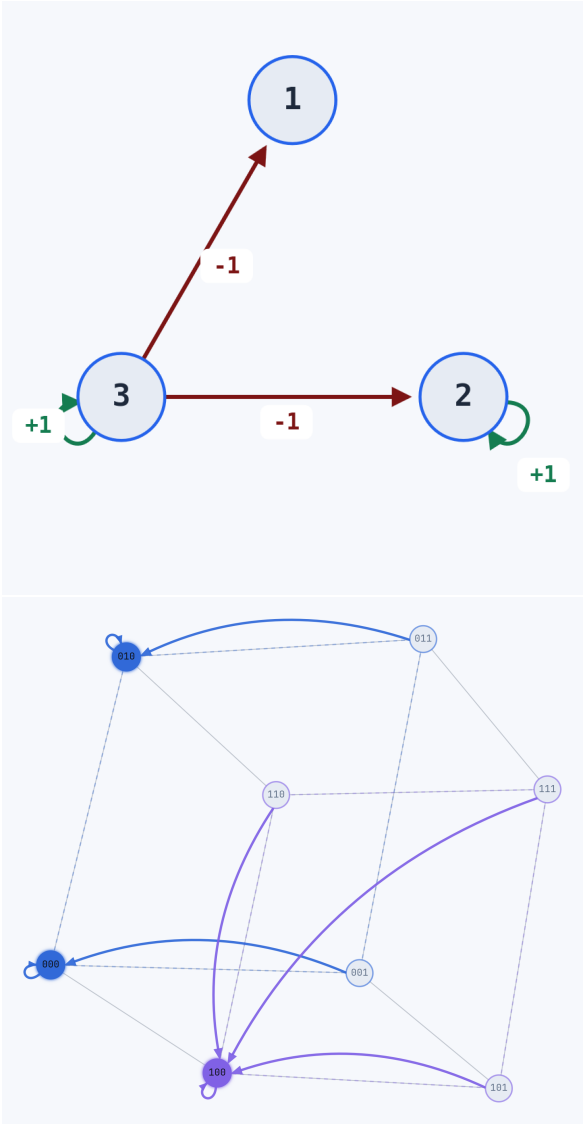
Le programme de Laurent travaille sur la dynamique : il énumère directement les fonctions booléennes des nœuds, sous forme de tables de vérité, en imposant que les faces désignées par l'arbre de décomposition soient invariantes. Ces fonctions sont ensuite restreintes à des SBF par des contraintes sur des poids. Ses appels ont d'abord été systématisés, et ses solutions exportées en CSV avec le script `export_decomposition.py`, pour les visualiser dans `sbn_viz` et mieux les apprécier. Ce programme génère les réseaux pluripotents à un nœud de contrôle par coupe, dont les sous-réseaux sont des SBN, pour un arbre de décomposition complet fixé, et dont les hyper-faces sont distinctes, c'est-à-dire que les deux faces de chaque coupe contiennent des sous-dynamiques différentes. Le calcul est cependant lourd et peu scalable, et il n'est utilisable en temps et ressources raisonnables, dans le cas général, que jusqu'en dimension 3 (Section 3.2).

3.1.2 Arbre fixé

Le rapport de Danwen prouvant qu'il est possible de caractériser cette famille de réseaux en ne travaillant que sur les poids, c'est dans cette direction que le travail s'est orienté, et il a abouti aux fichiers suivants : `PTBN_undercontrol.lp` génère les réseaux pluripotents d'arbre de décomposition fixé, en imposant à chaque nœud de coupe les conditions de Danwen (Section 2.3) sans rien demander aux sous-réseaux, qui sont donc des TBN à cause du contrôle. L'arbre de chaque vecteur y est pris dans une seule orientation, les coupes se faisant selon les nœuds d'indice le plus élevé (le nœud 3, puis le nœud 2 dans ses faces, en dimension 3). `PSBN_undercontrol.lp` étend le programme précédent en imposant que les sous-réseaux soient bien des SBN, par l'existence des poids w' de la Section 2.3 (Figure 10). Sur $\langle 0, 2, 0, 0 \rangle$ en dimension 3, ce dernier trouve 225 dynamiques, qui sont les 189 de Laurent et 36 dynamiques dont les deux faces portent la même sous-dynamique (Figure 11).



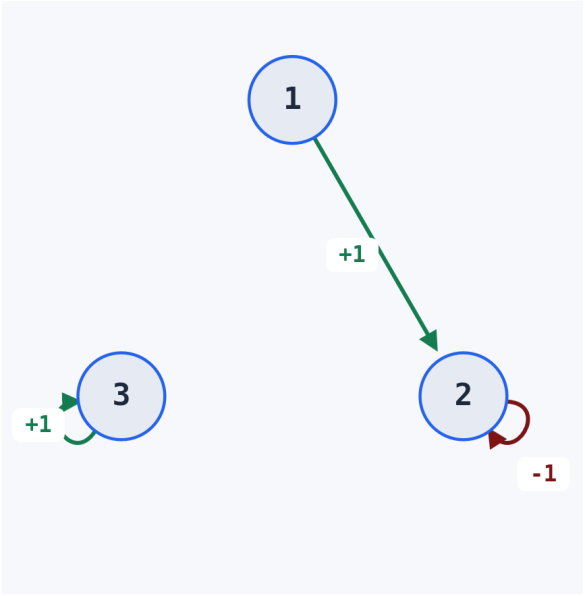
(a) Un réseau dont le sous-réseau de la face $x_3 = 1$ n'est pas un SBN, le nœud 2 s'y allumant depuis l'état 100.



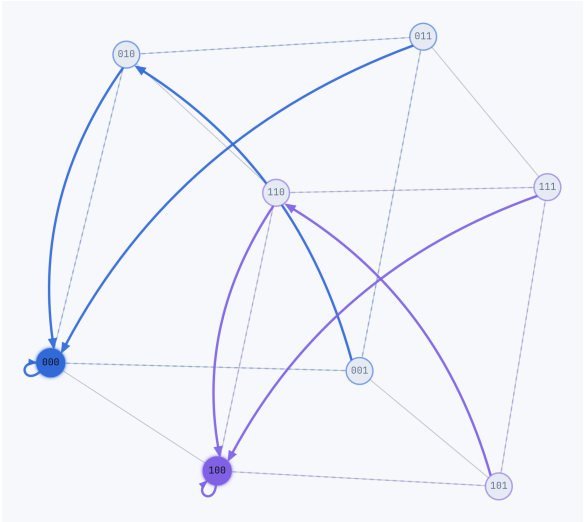
(b) Un PSBN proche du précédent.

FIGURE 10 – Deux réseaux de $\langle 0, 2, 0, 0 \rangle$, trouvés par `PTBN_undercontrol.lp` (à gauche) et par `PSBN_undercontrol.lp` (à droite).

La propriété d'avoir différentes hyper-faces a semblé moins essentielle (la grille de lecture de ce travail est au niveau des dynamiques, pas des sous-dynamiques, et un réseau qui simule deux fois le même sous-réseau n'a pas les mêmes potentialités, ou peut les utiliser pour deux usages différents) au regard des rendus prévus par la suite, et a donc été laissée de côté. C'est un choix. (`PSBN_distinct_faces_HS.lp` a quand même été développé dans l'idée de confirmer que les résultats obtenus correspondaient à ceux des programmes initiaux, ce qui a été le cas pour une seule configuration, $\langle 0, 2, 0, 0 \rangle$ en $d = 3$).



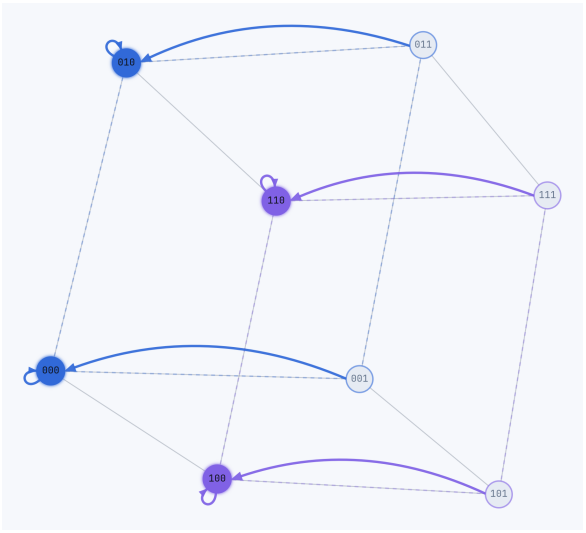
(a) La structure.



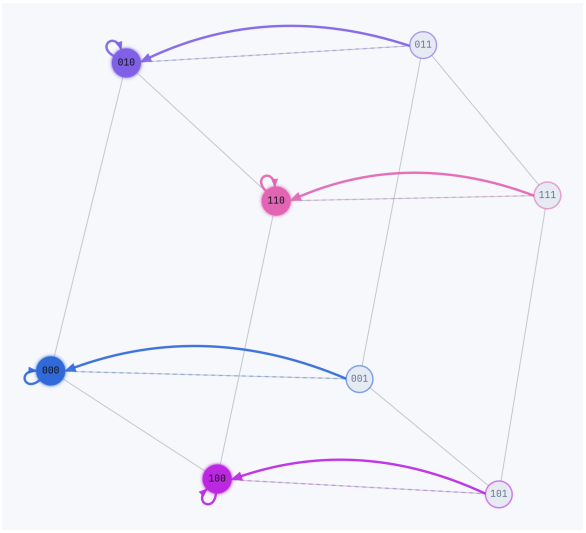
(b) Sa dynamique, coloriée par la décomposition.

FIGURE 11 – Une dynamique de $\langle 0, 2, 0, 0 \rangle$ que le programme de Laurent écarte, ses deux faces $x_3 = 0$ et $x_3 = 1$ portant la même sous-dynamique.

Une dynamique décomposable selon un arbre l'est souvent aussi selon un arbre plus fin, et elle apparaît alors sous plusieurs vecteurs (Figure 12), ce qui fausse les comptes et les visualisations par vecteur. L'objectif de `PSBN_undecomposablePBN_beyond.lp` était de n'énumérer chaque dynamique que sous son vecteur le plus fin, en interdisant aux feuilles de l'arbre de se décomposer davantage. Ce fut un relatif échec : des dynamiques étaient perdues (plus de 2 000 sur 32 768 en dimension 3), celles qui ne se décomposent que selon une autre orientation de l'arbre que celle fixée pour leur vecteur. Ce travail n'a pas été prolongé, car en termes de calcul il ne retirait qu'une petite partie de ce qu'il y a à énumérer, le nombre de dynamiques décomposables diminuant combinatoirement avec la finesse de décomposition.



(a) Coupée selon le nœud 3 ($\langle 0, 2, 0, 0 \rangle$).

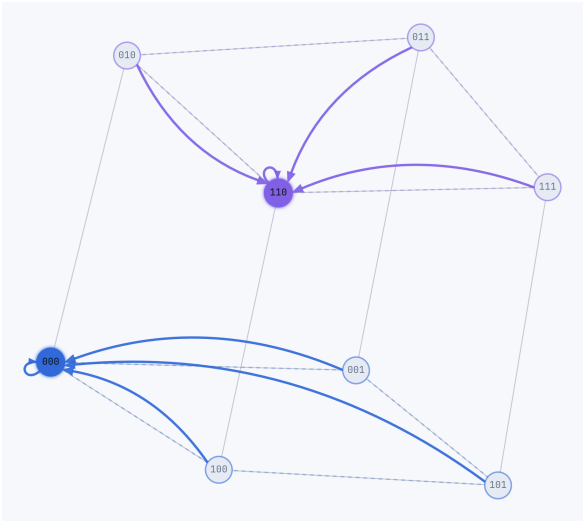


(b) Coupée selon le nœud 3 puis le nœud 2 ($\langle 0, 0, 4, 0 \rangle$).

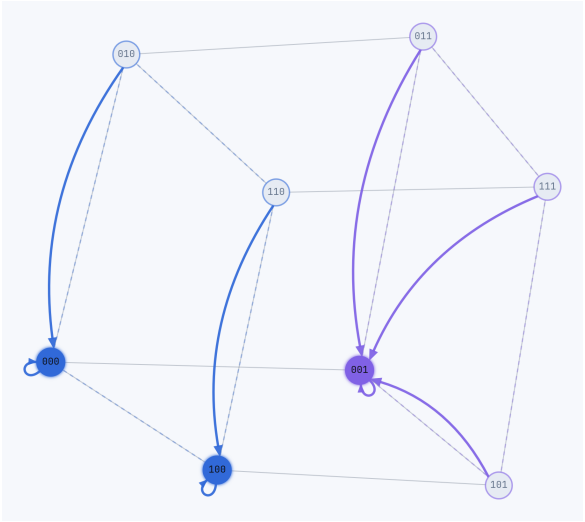
FIGURE 12 – Une même dynamique, énumérée sous le vecteur $\langle 0, 2, 0, 0 \rangle$ alors qu'elle admet une décomposition plus fine.

3.1.3 Arbre cherché par le solveur

L'étape suivante a été de laisser le solveur chercher l'arbre lui-même. `PTBN_undercontrol_treeinsol` accepte l'arbre de décomposition d'un vecteur donné quelle que soit l'orientation de l'hypercube, mais pas celle de ses sous-arbres. Il prend ainsi en compte les permutations des nœuds du réseau principal, mais pas celles des nœuds des sous-réseaux. `PSBN_undercontrol_treeinsolv` l'étend, de la même manière qu'avec les programmes à arbre fixé, en imposant que les sous-réseaux soient des SBN. Sur $\langle 0, 2, 0, 0 \rangle$ en dimension 3, ils trouvent les 658 PSBN du Tableau 2, contre 225 à arbre fixé (Figure 13, à droite), ainsi que des dynamiques qui ne se décomposent qu'en TBN et que l'arbre fixé manquait aussi (à gauche).



(a) Coupée selon le nœud 2, décomposable en TBN seulement.

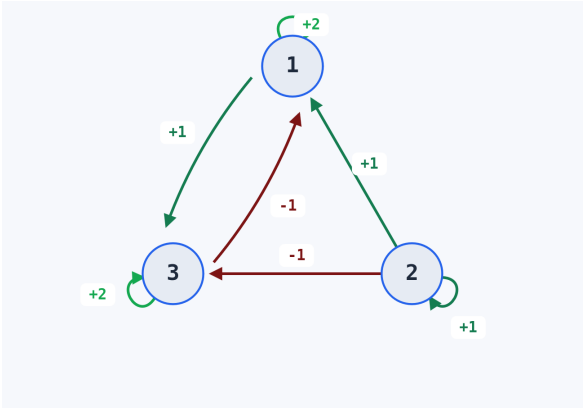


(b) Coupée selon le nœud 1, décomposable en SBN.

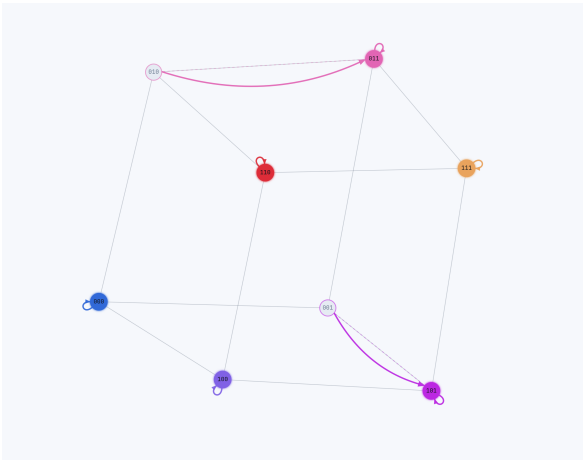
FIGURE 13 – Deux dynamiques de $\langle 0, 2, 0, 0 \rangle$ que seule une autre orientation de l'arbre révèle.

3.1.4 Arbre étiqueté

`PTBN_undercontrol_taggedtree.lp` et sa variante `PSBN_undercontrol_taggedtree.lp`, la version aboutie des programmes (Annexe A.3), prennent enfin en compte les permutations des nœuds des sous-réseaux. Il s'agit cette fois de chercher un arbre dont chaque coupe est étiquetée par le numéro de son nœud de contrôle, et les deux moitiés d'une coupe peuvent alors être recoupées selon des nœuds différents (Figure 14). Chaque dynamique y est énumérée une fois par arbre qu'elle admet, et son vecteur le plus fin s'obtient donc après coup. La question des décompositions plus fines, sur laquelle `PSBN_undecomposablePBN_beyond.lp` avait échoué, est ainsi contournée.



(a) La structure.



(b) Sa dynamique, coloriée par la décomposition.

FIGURE 14 – Une dynamique de $\langle 0, 0, 2, 4\rangle$ coupée selon le nœud 2, puis selon le nœud 1 dans la moitié $x_2 = 0$ et selon le nœud 3 dans la moitié $x_2 = 1$, ce que seul l'étiquetage des coupes permet d'exprimer.

3.1.5 Validation

La validation des programmes, par les visualisations, par le comptage des lignes et par la comparaison des fichiers produits, a tenu une place très importante dans ce travail, et les résultats se recoupent ainsi de plusieurs façons indépendantes. Les poids de chaque solution donnent bien ses fonctions de transition, et les nombres de dynamiques qui ont un nœud de contrôle ou qui se décomposent en SBN coïncident avec ceux du comptage par colonnes de la Section 2.3. Les 189 dynamiques de Laurent sont retrouvées sur $\langle 0, 2, 0, 0\rangle$, mais pas ses résultats sur les autres vecteurs (par exemple 11 dynamiques contre 6 sur $\langle 0, 1, 2, 0\rangle$). Il semble que la question de l'arbre de décomposition se soit mêlée à celles de l'énumération et des faces distinctes, et comme elle n'avait pas été envisagée à l'origine, la condition de faces distinctes ne se comporte plus de la même façon sur des arbres plus profonds que celui de $\langle 0, 2, 0, 0\rangle$. MCSBN, écrit indépendamment de l'ASP, retrouve en dimension 3 les mêmes vecteurs et les mêmes statistiques que JPP, et ces statistiques coïncident en dimension 2 avec un recalcul par force brute sur les 81 matrices de poids.

3.2 Optimisation

Du côté de l'ASP, les contraintes sur les poids se sont révélées beaucoup plus efficaces que celles sur les fonctions booléennes qu'emploie le programme de Laurent. Sur le seul cas où son énumération est retrouvée exactement, les 189 dynamiques de $\langle 0, 2, 0, 0\rangle$ en dimension 3 (Section 3.1), `PSBN_distinct_faces_HS.lp` les trouve en 0,014 s contre 2,5 s pour son programme, soit environ 180 fois plus vite, et la différence de machines (son temps a été mesuré sur la sienne) ne suffit pas à expliquer un tel écart. La formulation compte

aussi beaucoup, comme le montre l'histoire de `PSBN_undercontrol_taggedtree.lp` (Tableau 3). Sa première version plaçait les sommes de poids de `clingcon` dans le corps des règles, et saturait la mémoire avant la moindre solution. Une version intermédiaire, jamais versionnée, les remplaçait par des atomes auxiliaires au prix de copies des poids. La version finale calcule dès le grounding quels nœuds appartiennent à chaque chaîne de coupes, sans copie, et se révèle cinq fois plus rapide que la précédente sur $\langle 0, 2, 0, 0 \rangle$. La base PTBN, réécrite de la même façon, passe de 17 s à 0,8 s sur l'énumération complète de dimension 3.

TABLE 3 – Trois formulations successives de `PSBN_undercontrol_taggedtree.lp`, en dimension 3.

Version	$\langle 0, 2, 0, 0 \rangle$ (675 lignes)	Lignes en 60 s (énumération complète)
Première	mémoire saturée	mémoire saturée
Intermédiaire	2,6 s	4 120
Finale	0,50 s	33 013

`clingcon` sait aussi paralléliser sa recherche, mais son rendement décroît vite avec le nombre de fils (Tableau 4). Pour passer à l'échelle, une pipeline Java (dossier `JPPipeline/`) a donc été bâtie sur la bibliothèque JPP de Rémi [2]. Elle parallélise le calcul à un plus haut niveau, et permet ainsi de mieux tirer parti des différents cœurs de la machine et de mieux gérer l'utilisation de la mémoire. JPP organise un calcul en une chaîne d'étapes, les processeurs, qui transforment chacun une donnée d'entrée en une donnée de sortie. Un ensemble de fils d'exécution choisit à chaque instant, selon les données disponibles, quel processeur exécuter, et plusieurs fils peuvent exécuter le même processeur sur des données différentes. La machine reste ainsi occupée sans qu'il soit nécessaire d'écrire soi-même du code concurrent.

Cette pipeline enchaîne cinq processeurs (Figure 15). Pour une dimension donnée, elle énumère les vecteurs de décomposition, confie chacun à `clingcon` avec le programme ASP choisi, fait passer chaque solution par une étape de minimisation des poids, restée l'identité (Section 2.4), calcule ses statistiques, puis l'écrit dans le CSV au fur et à mesure.

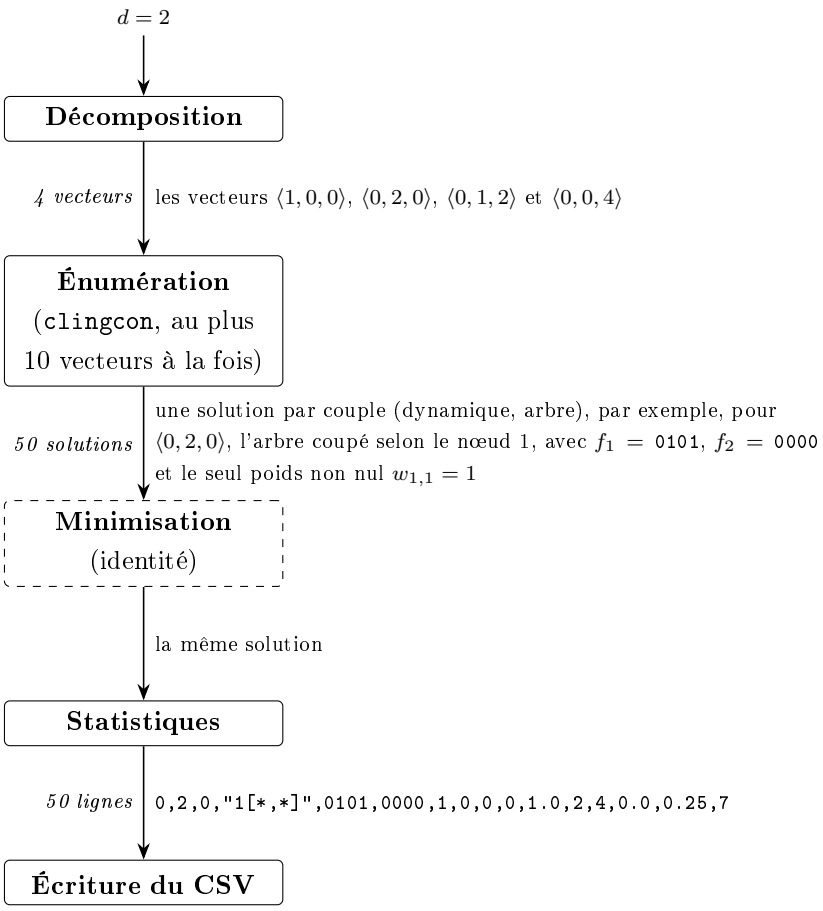


FIGURE 15 – La pipeline et ce qui circule entre ses étapes, en dimension 2 avec `PSBN_undercontrol_taggedtree.lp`.

L'énumération d'un vecteur passe par un garde-fou (Figure 16), dont l'utilité est d'éviter les trop gros appels à `clingcon`, qui pouvaient saturer la mémoire. JPP demande d'abord au plus s solutions à `clingcon`, et s'il en obtient moins, le vecteur est traité directement. Sinon, le vecteur est découpé en tranches, chacune fixant les valeurs de f_1 dans les états 0 à $d-1$, parmi les seuls préfixes réalisables par des poids, soit 2, 4, 6, 12 et 18 tranches pour d allant de 2 à 6. Chaque tranche est un appel indépendant, dont les solutions partent une à une vers la suite de la pipeline sans être accumulées, et la mémoire reste ainsi bornée. Un garde-fou récursif, qui découperait à son tour une tranche trop grosse, a été envisagé, mais il n'a finalement pas été nécessaire. L'usage de la pipeline est décrit en annexe (Annexe A.4).

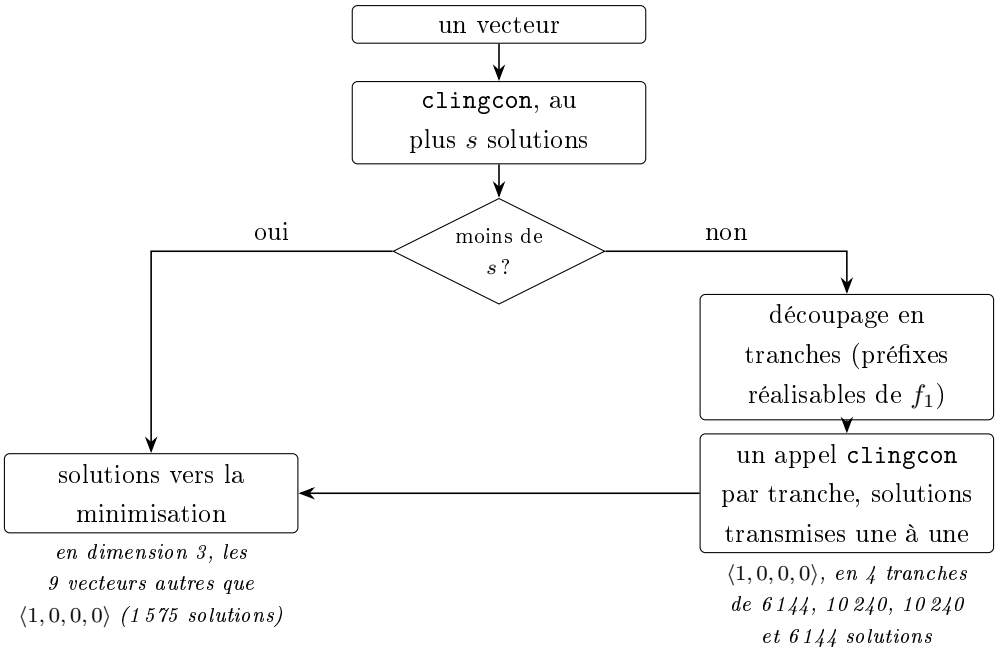


FIGURE 16 – Le garde-fou de l'énumération, avec les comptes de `PSBN_undercontrol_taggedtree.lp` en dimension 3 et $s = 1\,000$.

Le Tableau 4 compare, sur l'énumération complète des PSBN de dimension 3 avec `PSBN_undercontrol_taggedtree.lp` (34 343 lignes), la parallélisation propre de `clingcon` et JPP, sur un même ordinateur portable à 8 cœurs physiques (16 logiques). Le rendement de `clingcon` décroît avec le nombre de fils, mais JPP va 17 fois plus vite qu'un seul fil. Ce gain dépasse largement le nombre de cœurs physiques, même en tenant compte de l'incertitude des mesures, alors que `clingcon` n'atteint que 4,4 avec 8 fils. Le seul parallélisme ne suffit donc pas à l'expliquer, et le découpage du problème en un appel par vecteur, ou par tranche, semble accélérer aussi le calcul lui-même.

TABLE 4 – Énumération complète des PSBN de dimension 3 (34 343 lignes), selon le moteur. Les durées de plus de 10 s, mesurées une fois, varient d'environ 20 % d'une exécution à l'autre.

Moteur	Durée	Accélération
`clingcon`, 1 fil	66 s	×1
`clingcon`, 2 fils	35 s	×1,9
`clingcon`, 4 fils	21 s	×3,2
`clingcon`, 8 fils	15 s	×4,4
JPP	3,8 s	×17

Le post-traitement en Python, qui calculait les statistiques après coup, a été relativement abandonné ici : les statistiques sont désormais calculées exactement dans la pipeline (Section 2.4). Une machine de calcul distante a aussi été utilisée, qui accélérait les exécutions d'un facteur d'environ 4 à 10, mais aucun benchmark précis n'a été fait.

3.3 Visualisation

Les trois pages HTML, à la racine du dépôt, lisent les CSV de JPP et de MCSBN (Annexe A.2), et c'est à travers elles que les résultats ont été regardés, pour vérifier la justesse des programmes autant que pour apprécier l'espace des réseaux. Les construire a posé beaucoup de questions de forme, de lisibilité (palette de couleurs, style des points, fonctions interactives, ordre d'affichage), de fluidité (WebGL, lecture des fichiers en flux, rafraîchissement partiel) et d'ergonomie (lecture des différentes conventions de CSV mises en place pendant le développement), mais aussi beaucoup de questions de fond : comment organiser la légende, quelles statistiques afficher, comment donner le plus d'information possible, faut-il des barres d'erreur, quelle décomposition mettre en valeur ?

`sbn_viz.html` (Figure 17) montre un réseau à la fois, avec sa structure (les arcs colorés selon le signe de leur poids), les tables de vérité de ses fonctions, ses statistiques, son arbre de décomposition et sa dynamique sur l'hypercube des états. Cet hypercube peut tourner, ses attracteurs sont mis en évidence et ses sous-cubes coloriés selon la décomposition. C'est elle qui a produit la plupart des figures de réseaux et de dynamiques de ce rapport.

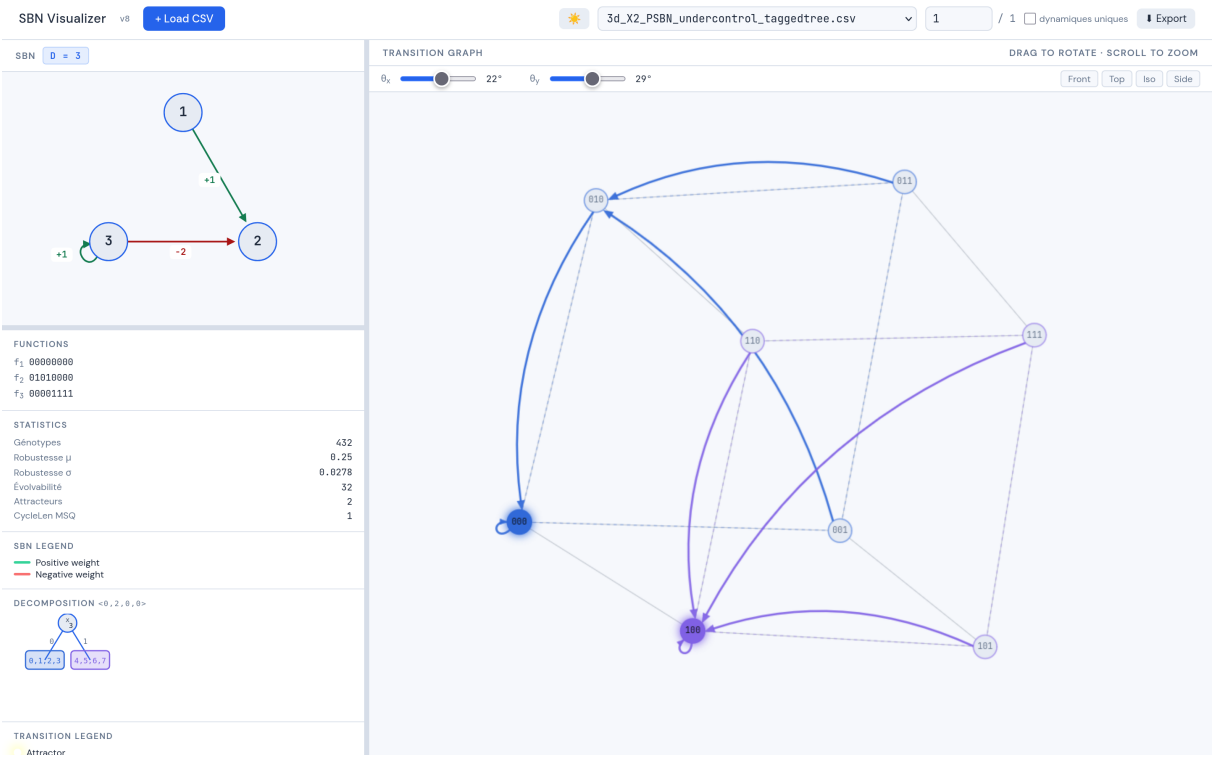


FIGURE 17 – La page `sbn_viz.html`, sur le réseau pluripotent de la Figure 2.

`sbn_scatter.html` (Figure 18) place les dynamiques dans le plan de deux statistiques choisies, par exemple la robustesse et l'évolutivité. Celles qui tombent au même endroit avec le même vecteur sont regroupées en un disque, dont la taille croît avec leur nombre de génotypes et dont la couleur indique le vecteur. La légende regroupe les vecteurs selon leur premier coefficient non nul, c'est-à-dire selon la dimension de leur plus grande hyper-face,

et les dynamiques les plus décomposées sont dessinées par-dessus les autres. Quand un axe est une moyenne, comme `Robustness_mean`, l'écart-type correspondant peut s'afficher en barres d'erreur. Un clic sur un disque donne la liste des dynamiques qu'il regroupe. C'est aussi la seule des trois pages qui se contente des statistiques, sans les fonctions ni les poids.



FIGURE 18 – La page `sbn_scatter.html`, sur les PSBN de dimension 3, avec les barres d'erreur et le détail d'un disque.

`metagraphe.html` (Figure 19) dessine en WebGL le métagraphe structurel de la Section 2.5, avec un nœud par dynamique distincte, relié aux dynamiques qu'une mutation permet d'atteindre. Les nœuds sont placés à l'avance par DRGraph [10] (Annexe A.5), qui procède par réduction de dimension, ou, à défaut, dans le navigateur par un algorithme force-directed classique (attraction le long des arêtes et répulsion entre tous les nœuds), les deux placements n'étant donc pas comparables. Les nœuds peuvent être colorés selon leur décomposition la plus fine, leur degré (c'est-à-dire leur évolutivité) ou une statistique. Un clic sur un nœud affiche ses statistiques, sa dynamique et sa structure, et fait ressortir ses voisins.

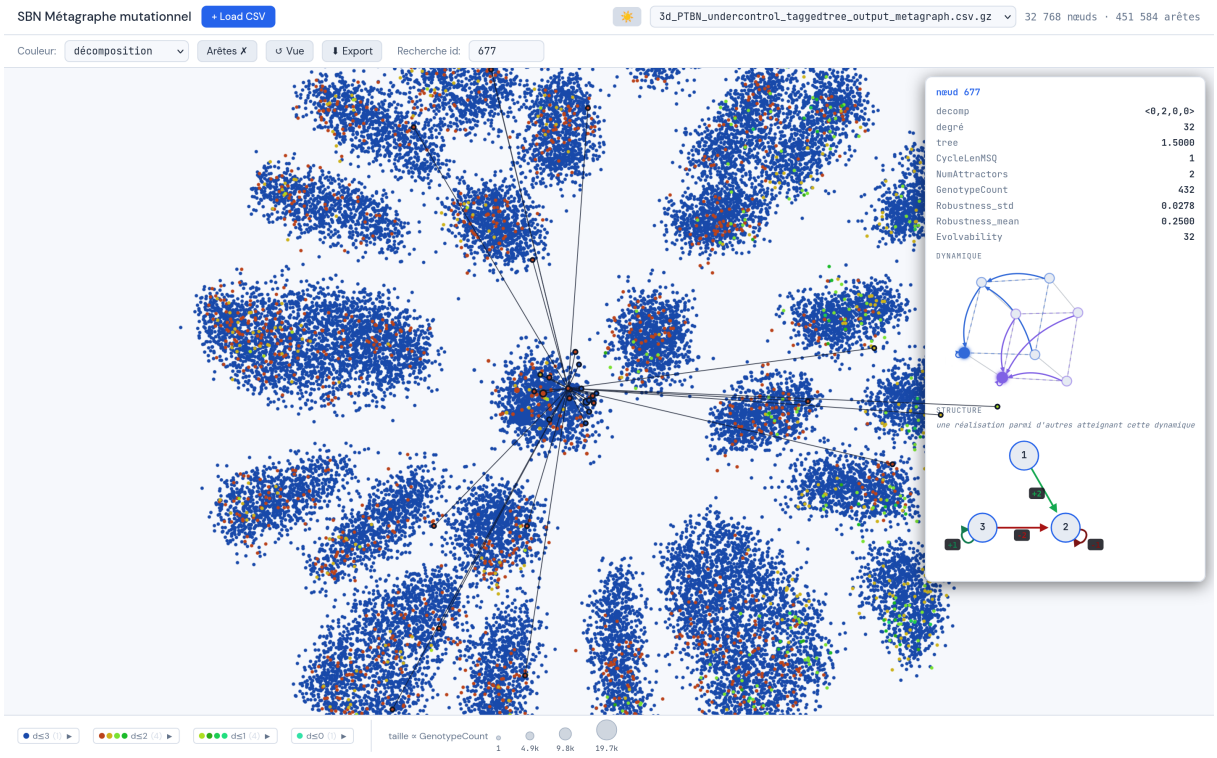


FIGURE 19 – La page `metagraphe.html`, sur le métagraphe de la Figure 9, après un clic sur un nœud.

L'outil de visualisation NVEIL, développé notamment par Pierre Jacquet, a aussi été essayé, mais les visualisations étaient alors déjà bien avancées, et les résultats qu'il donnait restaient relativement similaires à ceux déjà obtenus. Il est intéressant de noter que NVEIL se base comme ces travaux sur l'ASP, et qu'il aurait permis de gagner beaucoup de temps s'il avait été utilisé à une phase plus précoce du développement.

3.4 Usage des grands modèles de langage en recherche

Il n'aurait clairement pas été possible de mener le projet aussi loin sans l'aide d'un LLM. Et comme c'est un outil relativement nouveau et parfois clivant, il est intéressant de faire un point sur son usage ici d'un point de vue personnel. À noter que c'est quasi exclusivement Claude qui a été utilisé, notamment les modèles Sonnet 4.6 et Opus 4.8.

En termes de développement, cela permet à la fois de produire beaucoup plus, et donc d'envisager des projets plus ambitieux dès le départ, comme par exemple un outil interactif de visualisation HTML/JavaScript alors que je n'ai pas de formation particulière en développement web. Il faut évidemment émettre des réserves : le code produit étant beaucoup moins maîtrisé, il fallait nécessairement être plus rigoureux dans les phases de tests, mais cela est aussi largement facilité par les LLM. Il faut aussi noter que l'usage dépend énormément de la partie du projet. Pour le développement ASP par exemple, il s'agissait surtout de mieux comprendre les possibilités du langage, et pour le JavaScript, surtout de

produire rapidement des scripts facilement testables. Tout cela entre, je suppose, dans ce qu'on pourrait appeler du *prompt engineering*, qui, d'après mon expérience, reste crucial pour obtenir des résultats pertinents : le LLM est amené à prendre des décisions qui sont souvent discutables, mais qui ont néanmoins très souvent le mérite de poser de bonnes questions. Il reste donc nécessaire d'avoir une réflexion critique sur ce qu'il propose, sans quoi on peut vite se retrouver dépassé. Et comme je pense qu'un travail, aussi merveilleux soit-il, n'a d'intérêt (humain) que s'il est compris (humainement), il nous appartient de nous mettre au niveau. En termes de réflexion, je fais le même constat : malgré des résultats impressionnants, il faut toujours s'efforcer de comprendre ce qui est proposé. Par exemple, l'idée de preuve pour la borne des poids B_d, même si j'aurais préféré l'avoir trouvée seul, vient de Claude, mais je ne l'ai incluse dans ce rapport que parce que je pense l'avoir comprise et que je suis aussi convaincu de sa justesse que si je l'avais produite.

Enfin, il faut réaliser que les modèles et pratiques progressent (ô combien) rapidement, et que même si mon ressenti est un instantané relativement naïf qui n'est pas complètement inintéressant, il n'est qu'une mesure parmi d'autres de la façon dont ces outils bouleversent notre manière de travailler. La répétition de mesures similaires dira, sur un temps plus long, quelles doivent être nos approches vis-à-vis de ces outils.

3.5 Exploration Randomisée : MCSBN

Le dossier `MCSBN/` (en C et en Python) est une tentative, menée en véritable vibe code (avec autorisation d'écriture sans demande de permission pour le modèle de langage), d'explorer le paysage sans passer par l'ASP, pour voir si c'est plus efficace et quelles en sont les limites. MCSBN tire des dynamiques au hasard, chaque fonction étant prise uniformément parmi les SBF distinctes, et toutes les dynamiques sont donc équiprobables. Il calcule ensuite lui-même leur décomposition et leurs statistiques, et retrouve exactement, en dimension 3 où il peut tout énumérer, les vecteurs et les statistiques de JPP. Ce tirage est rapide (100 000 dynamiques en moins de 2 s en dimension 5), mais il ne récupère quasiment que des dynamiques non décomposées, même en TBN, 98,9 % en dimension 4 et 99,97 % en dimension 5. Il donne donc un bon échantillon des dynamiques, mais presque rien sur les pluripotentes. Le script `core_then_walk.py` prend le problème à l'envers, en énumérant un cœur de dynamiques très décomposées puis leur voisinage par marche aléatoire. Le paysage obtenu permet au métagraphe d'avoir des arêtes, mais il est biaisé vers la pluripotence, et illustre bien que la manière de générer est déterminante pour ce que l'on observe. DRGraph [10] sert enfin à précalculer rapidement le placement du métagraphe (Section 3.3), en une dizaine de secondes pour les 32 768 dynamiques de la dimension 3.

4 Résultats

Un des objectifs du projet étant son utilisation par autrui, j'essaie de présenter ici les moyens de (~~s'amuser avec~~) s'en servir sans avoir à s'y plonger trop avant.

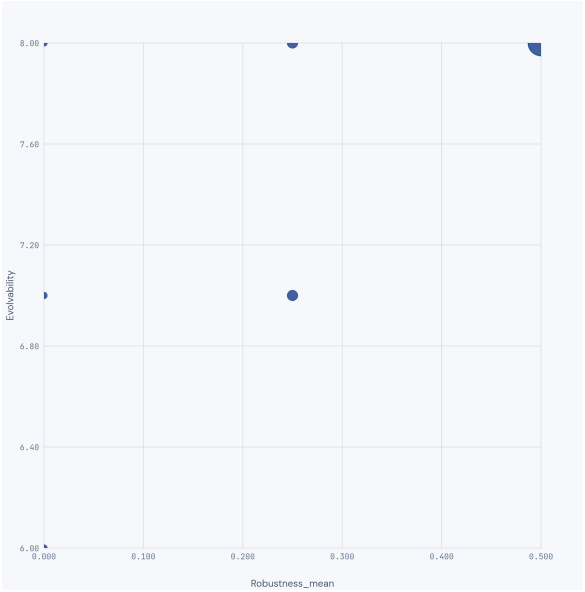
Tout ce qu'il faut pour générer les CSV est réuni en Annexe A. L'annexe explique comment lancer la pipeline et en changer les arguments, en ligne de commande ou par le `launch.json` de VS Code, comment modifier le garde-fou ou le nombre d'énumérations menées en parallèle, codés en dur, et comment ajouter une statistique dans `Statifier.java` (Annexe A.4). Elle précise aussi le format de sortie que doit garder un autre programme ASP pour être branché sur la pipeline (Annexe A.3), et l'usage de MCSBN (Annexe A.5). Les pages HTML lisent aussi les CSV compressés en gzip, ce qui vaut la peine pour les gros fichiers (Annexe A.1).

Dans un premier temps, il est rassurant d'observer que nos graphiques retrouvent les résultats de Wagner [7]. Les SBN possédant la même propriété que les séquences d'ARN sur les fréquences d'apparitions de phénotypes, on retrouve bien une corrélation positive entre évolutivité et robustesse phénotypiques.

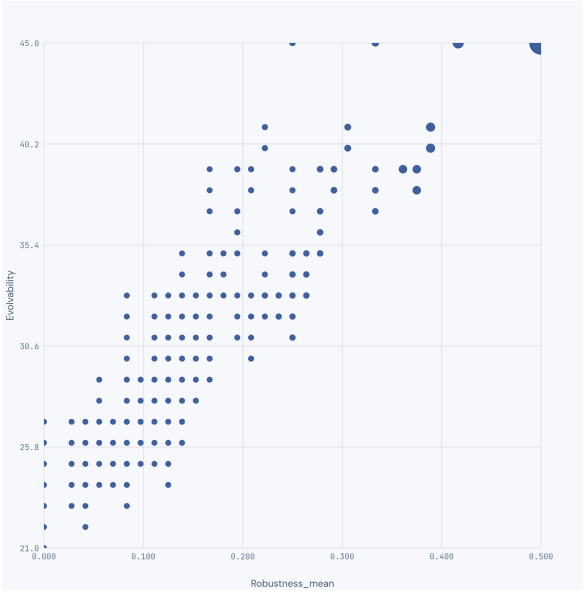
La corrélation de rang de Spearman entre la robustesse et l'évolutivité des dynamiques, la statistique qu'emploie Wagner, est positive et très significative dans chacune des dimensions étudiées (Tableau 5 et figure 20), et du même ordre que le $\rho = 0{,}55$ ($p < 10^{-17}$) qu'il obtient sur l'ARN. Le calcul porte sur toutes les dynamiques en dimension 2 et 3, et sur 100 000 dynamiques tirées uniformément par MCSBN (Section 3.5) en dimension 4 et 5.

TABLE 5 – Corrélation de Spearman entre la robustesse et l'évolutivité des dynamiques.

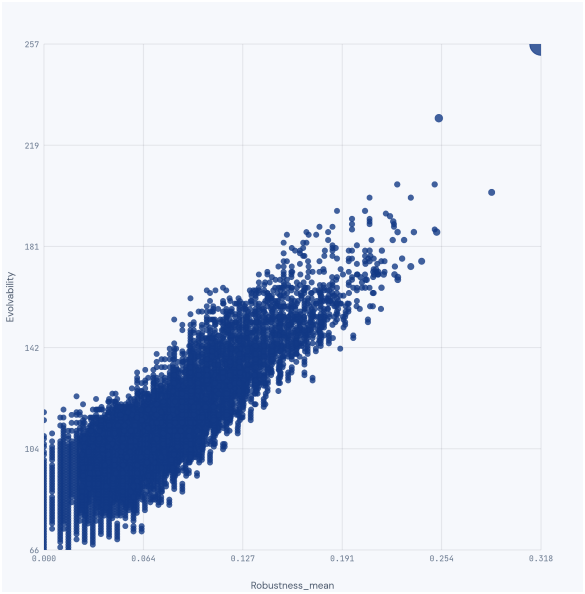
d	Dynamiques	ρ	p
2	36 (toutes)	0,62	$5 \cdot 10^{-5}$
3	32 768 (toutes)	0,81	$< 10^{-300}$
4	100 000 (tirées)	0,78	$< 10^{-300}$
5	100 000 (tirées)	0,69	$< 10^{-300}$



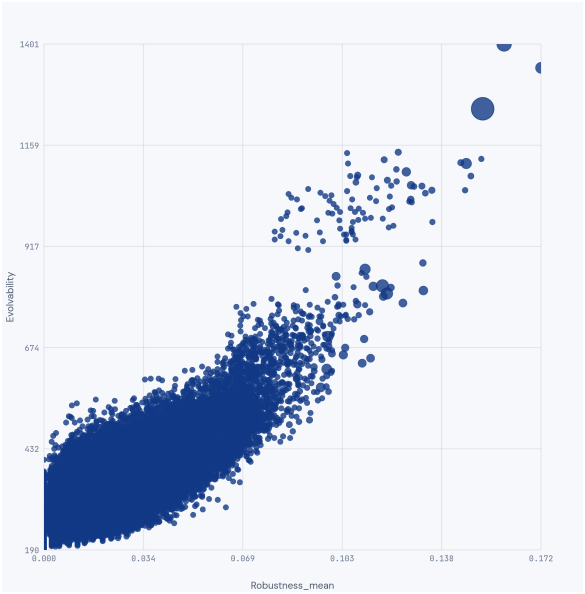
(a) $d = 2$



(b) $d = 3$



(c) $d = 4$ (tirage uniforme)



(d) $d = 5$ (tirage uniforme)

FIGURE 20 – Évolutivité en fonction de la robustesse des dynamiques, en dimension 2 à 5.

Malgré nos limitations à des dimensions faibles, il est possible d'observer des tendances dont l'explication pourrait éventuellement permettre de généraliser aux dimensions supérieures.

- La décomposition d'un SBN en sous-dynamiques distinctes est déterminante dans le calcul de ses autres propriétés. (La pluripotence n'est pas décorrélée de tout) (Figure 21).



FIGURE 21 – Évolutivité en fonction de la robustesse en dimension 3, colorée selon le vecteur de décomposition.

- Les propriétés d'un PSBN semblent être très liés à la dimension de sa plus grande hyper-face non décomposable. Notamment la variance de la robustesse génotypique pour chaque phénotype (Robustness_std) (Figure 22).



FIGURE 22 – Écart-type de la robustesse en fonction de sa moyenne en dimension 3, coloré selon le vecteur de décomposition.

- Des SBF particulières forment des groupes dans le graphique de l'écart-type contre la moyenne de la robustesse, typiquement la fonction constante 0 (Figure 23). En dimension 3, le nombre k de nœuds dont la fonction est la constante 0 range presque les dynamiques selon leur robustesse moyenne : de 0 à 0,25 pour $k = 0$, de 0,17 à 0,33 pour $k = 1$, de 0,33 à 0,42 pour $k = 2$, et 0,5 pour l'unique dynamique où $k = 3$. Cela s'explique par la robustesse de la fonction constante 0, qui vaut $1/2$ (ses réalisations sont les colonnes de poids tous négatifs ou nuls, et la moitié des mutations de chacun de ces poids le laissent négatif ou nul), et aucune autre SBF ne dépasse $1/4$. La robustesse d'une dynamique étant la moyenne de celles de ses colonnes, chaque nœud constant à 0 la tire donc vers le haut. L'écart-type moyen baisse aussi quand k augmente (0,022 ; 0,017 ; 0,010 ; 0).

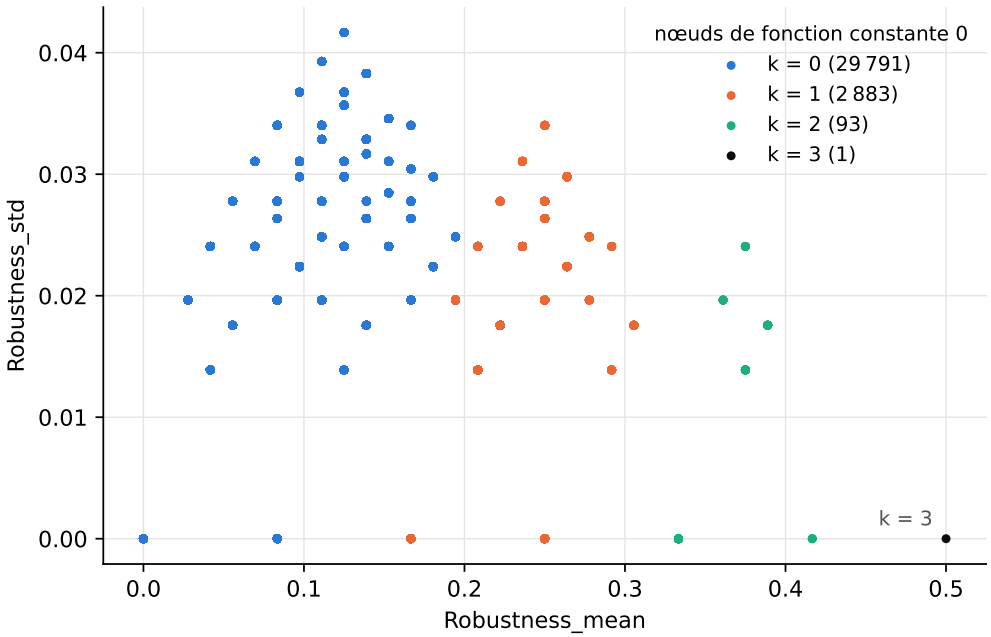


FIGURE 23 – Écart-type de la robustesse en fonction de sa moyenne en dimension 3, selon le nombre k de nœuds dont la fonction est la constante 0.

Dans le contexte de mutations sur les poids (Section 2.5) :

- Les PTBN sont plutôt proches les uns des autres, notamment au sein d'une même orientation (élément d'une orbite de permutation). Les PTBN les plus décomposés (que les permutations affectent moins, Figure 9) font en quelque sorte le lien entre ces différentes orientations.

Ces conclusions résument les observations sur les visualisations générées, tentent d'en trouver des explications qui nous permettent d'oser une généralisation, mais peuvent difficilement dépasser en précision les visualisations elles-mêmes. C'est pourquoi le lecteur est fortement invité à les tester par lui-même (et qui sait, peut-être trouver de tendances que l'on aurait manquée).

5 Conclusion, travaux inachevés et ouverture

L'objectif principal était de montrer ce que le formalisme des SBN permettait. Et à cet égard le présent travail semble nous indiquer qu'il parvient assez bien à donner des interprétations claires de concepts généraux (comme la pluripotence), et qu'il est suffisamment pratique et riche pour que l'exploration de ses possibilités soit faisable et porteuse de sens.

Plusieurs travaux, évoqués au fil du rapport, sont restés inachevés, et il est utile de les rassembler ici pour qui voudrait les reprendre.

- Affichage d'arbre généalogique issu des simulations, qui dessinerait dans le méta-

graphe les chemins d'évolution des agents de Corentin (Section 2.5).

- Minimisation de poids : elle est restée l'identité dans la pipeline, mais une réalisation canonique de chaque dynamique permettrait des statistiques sur la structure elle-même (Section 2.4).

- PluripotentSignedBooleanNetworkInhibiteur : la piste n'existe qu'avec un arbre fixé, mais ces réseaux sont, en dimension 3, beaucoup plus souvent pluripotents que les autres (Section 2.3).

- Exploration dirigée (MCSBN) : un tirage uniforme ne donne presque aucun réseau pluripotent, et le paysage hybride est biaisé (Section 3.5).

- Meilleure FrontEnd (il sera toujours améliorable), par exemple pour colorer le méta-graphe par orientation ou montrer les sous-cubes invariants d'une décomposition partielle.

Au-delà de ces travaux, quelques idées pourraient prolonger le sujet.

- Gestion des arêtes non existantes (et pas juste de poids nuls), à la manière des SBF innées et projetées de Rémi (Section 2.2), avec des règles de mutation qui créent ou suppriment des arcs.

- Comparaison des dimensions entre elles, par exemple à travers des règles de mutation qui ajoutent ou retirent des nœuds (Section 2.5).

- Explorer d'autres aspects de la pluripotence : noeuds contrôlés, offuscation et vecteurs de décomposition partiels (Section 2.3).

- Exploitation plus aboutie des résultats, dont les tendances de la Section 4 reposent pour la plupart sur la lecture des graphiques et demanderaient à être mesurées.

Références

[1] Nicolas Glade. "Le Vivant Rare, Faible et Amorphe — Évolution depuis les Origines jusqu'à la Vie telle qu'elle nous Apparaît". HAL : tel-04075075. Habilitation à Diriger des Recherches. Université Grenoble Alpes, 2022.

[2] Rémi Segretain. "Complexité et évolution : étude des Réseaux Booléens à Signe". École doctorale ISCE, spécialité MBS. Directeur : Nicolas Glade. Thèse de doctorat. Université Grenoble Alpes, 2023.

[3] Rémi Segretain et al. "Implementation of a Computing Pipeline for Evaluating the Extensibility of Boolean Networks' Structure and Function". In : (2020). bioRxiv preprint, doi :10.1101/2020.10.02.323949.

[4] Ai-Ling Nguyen Bonnet. *Évolution des réseaux booléens à signe*. Rapport de stage M1. Encadrants : Sergiu Ivanov et Nicolas Glade. Université Grenoble Alpes, 2023.

[5] Rémy Defossez. *Le mille-feuille évolutionnaire, une nouvelle formalisation de l'évolution*. Rapport de stage L3. Encadrants : Sergiu Ivanov et Nicolas Glade. ENS Paris Ulm / Université Grenoble Alpes, 2021.

[6] Danwen Wu. *Formalization, identification and decomposition of pluripotents Boolean networks*. Rapport de stage M1. Encadrants : Nicolas Glade et Laurent Trilling, TIMC Laboratory. Université Grenoble Alpes, 2024.

[7] Andreas Wagner. "Robustness and evolvability : a paradox resolved". In : *Proceedings of the Royal Society B* 275.1630 (2008), p. 91-100. doi : 10.1098/rspb.2007.1137.

[8] Mutsunori Banbara et al. "Clingcon : The Next Generation". In : *Theory and Practice of Logic Programming* (2017). arXiv :1705.04569.

[9] Mohimenul Kabir et al. "ApproxASP — a Scalable Approximate Answer Set Counter". In : *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)*. 2022, p. 5755-5764.

[10] Minfeng Zhu et al. "DRGraph : An Efficient Graph Layout Algorithm for Large-scale Graphs by Dimensionality Reduction". In : (2020). arXiv :2008.07799.

A Guide d'utilisation des programmes

Cette annexe explique comment utiliser les programmes au point, avant tout la pipeline JPP, et dans une moindre mesure MCSBN. Ses deux premières parties réunissent ce qu'il faut savoir avant de s'en servir, à savoir où ils se trouvent dans le dépôt, comment installer leurs dépendances, et sous quel format ils écrivent les CSV que lisent les visualisations, tandis que les suivantes détaillent l'usage de chacun d'eux.

A.1 Organisation du dépôt et installation

La figure 24 montre quel programme produit quoi et qui le lit, et le tableau 6 indique où se trouve chacun d'eux dans le dépôt, ainsi que la partie de cette annexe qui en détaille l'usage.

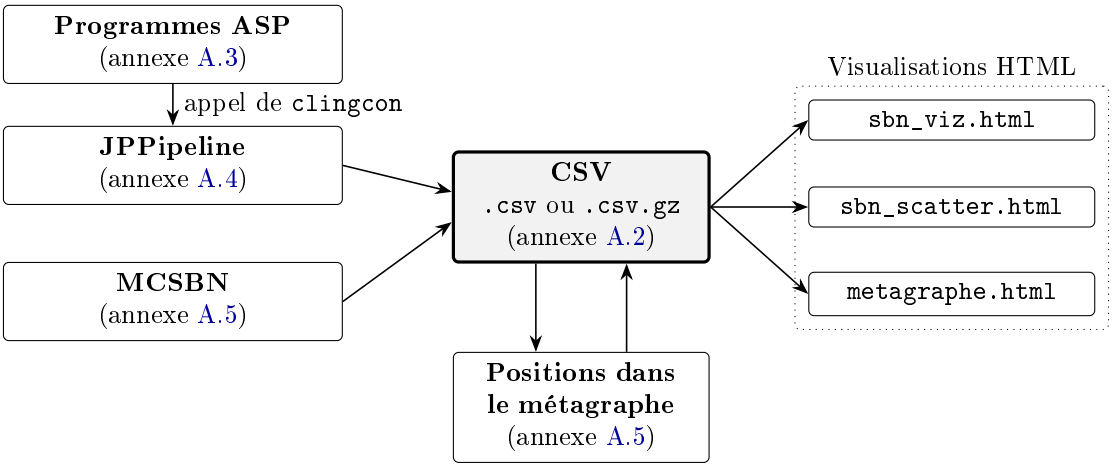


FIGURE 24 – Chaîne de traitement. Le calcul des positions dans le métagraphe est facultatif : il relit un CSV et le réécrit avec deux colonnes de plus.

JPP et MCSBN écrivent des CSV au même format, à la colonne `tree` près, que l'on peut compléter par les positions des dynamiques dans le métagraphe avant de les ouvrir dans les pages HTML, lesquelles fonctionnent directement dans le navigateur et lisent aussi les fichiers compressés en gzip. Le dépôt ne contient pas ces CSV, que le `.gitignore` exclut comme les binaires compilés, si bien qu'il faut les régénérer ou se les procurer à part.

Le fichier `environment.yml`, à la racine du dépôt, décrit l'environnement conda du projet, qui réunit les dépendances peu courantes (`clingcon`, les bibliothèques GSL et Boost qu'utilise DRGraph, et celles des scripts Python) ; on le crée puis l'active depuis la racine, ce qui met `clingcon` dans le `PATH`, où JPP va le chercher :

```
conda env create -p .conda -f environment.yml && conda activate ./.conda
```

Une passe complète consiste alors à lancer JPP pour chaque couple (programme ASP, dimension) puis MCSBN, à compresser les gros CSV avec `gzip` et à ajouter au besoin les

Partie	Emplacement	Contenu
A.3. Programmes ASP	`JPPipeline/src/asp/`	Programmes d'énumération et leurs fichiers d'appui (`setup_btree_{1..6}d.lp`, `Subdivision.lp`).
A.4. JPPipeline	`JPPipeline/src/java_src/`, `JPPipeline/lib/jpp.jar`, `.vscode/launch.json`	Pipeline Java, bibliothèque JPP de Rémi [2] et configurations de lancement ; les CSV sont écrits dans `JPPipeline/out/`.
A.5. MCSBN	`MCSBN/`	Générateur Monte-Carlo en C et en Python, et placement du métagraphe par DRGraph (`tools/DRGraph/`).
Visualisations	`html_visualization_lib/`, `*.html` à la racine	Sources des trois pages, et leurs versions autonomes, à régénérer avec `build_standalone.py` après toute modification des sources.
A.6. Scripts annexes	`SBN_pluripotent/`, `SBN_pluripotent_Laurent/`, `JPPipeline/src/python/`	Calcul des bornes des poids, versions antérieures des programmes et post-traitements abandonnés.

TABLE 6 – Emplacement dans le dépôt des programmes décrits dans chaque partie de cette annexe.

positions du métagraphe.

Le dépôt contient bien d'autres programmes que ceux référencés ici (essais, versions antérieures, outils abandonnés en cours de route), et rien de ce qui n'est pas présenté comme fonctionnel dans cette annexe n'est garanti sans s'être penché sur le code pour le comprendre. Il en va de même des données produites avant la rédaction de cette annexe, dont certaines suivent d'anciennes conventions (noms de colonnes, fonctions écrites en entier, autre règle de mutation) et qu'il vaut mieux régénérer que comparer aux nouvelles.

A.2 Format des CSV

Chaque CSV commence par une ligne d'en-tête, et comme les pages HTML repèrent les colonnes par leur nom, des colonnes en plus ne les gênent pas. Le tableau 7 donne ces colonnes dans l'ordre où JPP les écrit, MCSBN écrivant les mêmes à l'exception de `tree`.

Les états sont numérotés de 0 à $2^d - 1$, le bit $i - 1$ de l'état s valant 1 quand le nœud i est actif (le nœud 1 est donc le bit de poids faible), et le caractère numéro s de `f_`j, en comptant depuis 0 à partir de la gauche, donne la valeur du nœud j dans l'état qui suit s. Les poids sont des entiers compris entre $-w_\mathrm{b}(d)$ et $w_\mathrm{b}(d)$, avec $w_\mathrm{b}(d) = 1, 1, 2, 3, 5$ et 6 pour d allant de 1 à 6 (en $d = 6$, cette valeur diffère de la borne du tableau 2).

La colonne `tree` écrit l'arbre de décomposition de façon imbriquée, en notant `*` une feuille (un sous-cube qui n'est plus découpé) et `N[A,B]` une coupe selon le nœud de contrôle `N`, où `A` est l'arbre de la moitié où `N` est inactif et `B` celui de la moitié où il est actif. Ainsi `1[2[*,*],*]` coupe selon le nœud 1 puis recoupe selon le nœud 2 la seule moitié où le nœud 1 est inactif, et l'on retrouve le vecteur en comptant les feuilles, v_k étant le nombre

Colonne	Contenu
`v_`d, …, `v_0`	Vecteur de décomposition $\langle v_d, \ldots, v_0 \rangle$, où v_k est le nombre de sous-cubes (hyper-faces) de dimension k aux feuilles de l'arbre.
`tree`	Arbre de décomposition étiqueté par les nœuds de contrôle, entre guillemets (notation ci-dessous) ; vide si le programme ASP ne le fournit pas.
`f_1`, …, `f_`d	Fonction de transition du nœud j, en table de vérité de 2^d caractères `0` ou `1`.
`"w_`i`,`j`"`	Poids de l'arc $i \to j$.
`CycleLenMSQ`	Moyenne, sur les attracteurs, du carré de leur longueur.
`NumAttractors`	Nombre d'attracteurs, points fixes et cycles.
`GenotypeCount`	Nombre de matrices de poids qui réalisent la même dynamique.
`Robustness_std`, `Robustness_mean`	Écart-type et moyenne de la robustesse de ces matrices.
`Evolvability`	Évolutivité de la dynamique.

TABLE 7 – Colonnes des CSV, dans l'ordre du fichier (d est la dimension). Les statistiques sont définies dans la section « Statistiques, propriétés, métriques ».

de feuilles à la profondeur $d - k$.

Une ligne ne correspond pas à un réseau mais à un couple (dynamique, arbre de décomposition), car les programmes `undercontrol` produisent une même dynamique une fois par arbre qu'elle vérifie, et le vecteur trivial $\langle 1, 0, \ldots, 0 \rangle$, qui n'impose aucune contrainte, les contient toutes ; le CSV $d = 3$ de `PSBN_undercontrol_taggedtree` compte ainsi 34 343 lignes pour seulement 32 768 dynamiques. Les statistiques d'une dynamique sont les mêmes sur toutes ses lignes, alors que ses poids peuvent varier d'une ligne à l'autre, les colonnes `w_*` ne donnant qu'une matrice parmi toutes celles qui la réalisent ; pour compter des dynamiques, il faut donc dédoublonner selon $(f_1, \ldots, f_d)$, comme le fait `metagraphe.html`.

Le calcul des positions dans le métagraphe ajoute deux colonnes en fin de ligne, `metagraph_x` et `metagraph_y`, que `metagraphe.html` utilise pour placer les dynamiques. À l'inverse, JPP peut omettre les colonnes `f_*` et `w_*` (dernier argument à 0), mais seule `sbn_scatter.html` peut alors exploiter le fichier. Les pages HTML lisent par ailleurs la dimension dans le préfixe du nom de fichier (`2d_`, `3d_`…), qu'il vaut donc mieux conserver.

A.3 Programmes ASP d'énumération

La version aboutie des programmes ASP est `PSBN_undercontrol_taggedtree.lp`, dans `JPPipeline/src/asp/`, ou sa variante `PTBN_undercontrol_taggedtree.lp` si l'on admet des sous-réseaux qui ne sont pas des SBN. Les autres programmes du dossier (familles `undercontrol`, `treeinsolver` et `undecomposablePBN_beyond`) sont des versions intermédiaires, dont l'objectif a été atteint ou non selon les cas (section « Correction »), mais qui n'ont pas été maintenues jusqu'au bout.

Le programme lit la dimension et le vecteur de décomposition dans des constantes, et

se lance seul avec

```
clingcon <n> --project -c d=<d> -c v<d>=<v_d> ... -c v0=<v_0> \
      JPPipeline/src/asp/PSBN_undercontrol_taggedtree.lp
```

`<n>` nombre de solutions à afficher (0 pour toutes) ;

`d` dimension, au plus 5 ;

`v<k>` nombre de sous-cubes de dimension k du vecteur de décomposition ; laissées à -1, leur valeur par défaut, ces constantes n'imposent aucun vecteur et le programme énumère tous les SBN de dimension d.

Chaque solution donne les atomes `transition_function_bit(N,S)`, présents quand le nœud `N` est actif dans l'état qui suit `S`, les poids `w(I,J)` de l'arc `I` $\to$ `J`, et les atomes `branch_node/4` et `leaf_node/3`, qui décrivent l'arbre de décomposition. C'est ce format que lit JPP (`ClingconUtil.java`), si bien qu'un nouveau programme doit le respecter pour être utilisé par la chaîne.

A.4 JPPipeline : énumération parallèle et statistiques

JPP se compile et se lance depuis la racine du dépôt, avec `clingcon` dans le `PATH`, que fournit par exemple l'environnement conda du projet (annexe A.1) :

```
javac -cp JPPipeline/lib/jpp.jar -d JPPipeline/bin \
      $(find JPPipeline/src/java_src/PSBN -name '*.java')
java -cp JPPipeline/bin:JPPipeline/lib/jpp.jar PSBN.Main \
      <dim> <lp_enumeratorFile> <setup_btreeFolder> <nSolutions> \
      <output_path> [includeWeightsAndTransitions]
```

`<dim>` dimension d des réseaux ;

`<lp_enumeratorFile>` programme ASP qui énumère les réseaux (annexe A.3) ;

`<setup_btreeFolder>` dossier contenant `setup_btree_<d>d.lp` et `Subdivision.lp`, en pratique `JPPipeline/src/asp` ;

`<nSolutions>` nombre maximal de solutions par vecteur de décomposition (0 pour toutes) ;

`<output_path>` dossier où écrire le CSV ;

`[includeWeightsAndTransitions]` facultatif, 0 pour omettre les colonnes `f_*` et `w_*` (1 par défaut).

Par exemple, la configuration `Main` de `.vscode/launch.json`, qu'on peut aussi lancer directement depuis VS Code, correspond à

```
java -cp JPPipeline/bin:JPPipeline/lib/jpp.jar PSBN.Main 2 \
      JPPipeline/src/asp/PSBN_undercontrol_taggedtree.lp JPPipeline/src/asp \
      0 JPPipeline/out 1
```

Le CSV, nommé `<dim>d_<programme>_output.csv`, s'écrit au fur et à mesure dans le dossier `<output_path>`, et il vaut mieux le compresser ensuite avec `gzip`, puisque les pages HTML lisent directement les fichiers `.csv.gz`. Pour suivre un long calcul, les options `-Djpp_monitoring=true` et `-Djpp_monitoring.stepTime=5`, placées avant `-cp` (elles figurent déjà dans `launch.json`), affichent toutes les cinq secondes l'avancement de chaque étape et la mémoire utilisée.

L'argument `<nSolutions>` permet de ne demander à `clingcon` qu'un nombre limité de solutions par vecteur. Comme `clingcon` est lancé avec une graine fixe, cette énumération partielle redonne les mêmes réseaux d'une exécution à l'autre. Lorsque ce nombre est nul ou dépasse 1000, un garde-fou arrête l'énumération d'un vecteur à 1000 solutions et la relance en tranches, en fixant les premiers bits de la fonction du nœud 1 grâce à `Subdivision.lp`, pour limiter la mémoire de chaque appel ; le nombre de solutions demandé est alors réparti entre les tranches. Ce seuil de 1000 solutions (`SUBDIVISION_THRESHOLD`) et le nombre maximal d'énumérations non subdivisées menées en parallèle (10, par `setMaximumParallelTasks`) sont codés en dur dans `Pipe.java`, où il faut les modifier pour les adapter à la machine, les autres étapes utilisant déjà autant de fils d'exécution que la machine a de cœurs. Limiter ainsi le nombre d'énumérations menées en parallèle évite d'engorger JPP.

Pour ajouter une statistique, on la calcule dans la méthode `process` de `Statifier.java`, qui reçoit chaque réseau et renvoie sa ligne de CSV, puis on ajoute son nom, à la bonne place, dans l'en-tête que construit la méthode `reset` de `Pipe.java` ; les statistiques qui portent sur les attracteurs sont, elles, calculées dans `SBN.toCsvRow`. Pour calculer la statistique, il est souvent possible de s'appuyer sur les tables par SBF de `SBFStatTable.java`, que `Statifier` calcule une fois par dimension : elles donnent, pour chaque SBF, le nombre de colonnes de poids bornés qui la réalisent, le nombre moyen de mutations neutres de ces colonnes et sa variance, et les SBF qu'une mutation permet d'atteindre. Comme les colonnes de W sont indépendantes, une statistique qui se décompose par colonnes s'obtient alors en combinant les valeurs de ces tables pour $f_1, \ldots, f_d$, sans parcourir les matrices de poids, ce que fait déjà `process` pour `GenotypeCount`, la robustesse et l'évolutivité.

A.5 MCSBN : exploration Monte-Carlo

MCSBN se compile et se lance depuis son dossier, `MCSBN/`, où il garde en cache, dans `.cache/`, les tables qu'il calcule à la première exécution dans chaque dimension :

```
make && make metalayout && make drgraph
```

Ces trois cibles produisent respectivement le générateur `bin/mcsbn`, le calcul des positions dans le métagraphe `bin/metagraph_layout` et le binaire `tools/DRGraph/Vis` de

DRGraph, qui va chercher GSL et Boost dans l'environnement conda (annexe A.1) ; le `Makefile` fonctionne aussi sous Windows, depuis le shell MSYS2 MinGW64.

Le générateur s'utilise ainsi :

```
./bin/mcsbn -d <d> [-n <n>] [-o <fichier>] [--measure variety|genotype]
      [--seed <s>] [--max-draws <m>] [--exhaustive] [--no-weights]
      [--progress|--no-progress]
```

`-d <d>` dimension, de 1 à 6 ;

`-n <n>` nombre de dynamiques distinctes à produire (100 000 par défaut) ;

`-o <fichier>` fichier de sortie, compressé si son nom finit par `.gz` (sortie standard par défaut) ;

`--measure variety` (par défaut) tire chaque fonction uniformément parmi les SBF distinctes, si bien que toutes les dynamiques sont équiprobables, alors que `genotype` la tire en proportion de son nombre de réalisations, comme un tirage uniforme des matrices de poids ;

`--seed <s>` graine du tirage, fixe par défaut, de sorte que deux exécutions identiques donnent le même fichier ;

`--max-draws <m>` nombre maximal de tirages, au-delà duquel le programme s'arrête même s'il n'a pas obtenu n dynamiques distinctes, ce qui n'arrive qu'en petite dimension ($50n$ par défaut) ;

`--exhaustive` énumère toutes les dynamiques au lieu de tirer, ce qui n'est accepté qu'en deçà de $5 \cdot 10^7$ dynamiques ;

`--no-weights` omet les colonnes `f_*` et `w_*` ;

`--progress, --no-progress` force ou coupe l'affichage de l'avancement, actif par défaut dans un terminal.

La version Python, `src/python/mcsbn.py`, accepte les mêmes options mais tourne bien plus lentement ; elle sert de référence aux tests.

Un tirage de `mcsbn` est trop clairsemé pour que le métagraphe ait des arêtes, et c'est `src/python/core_then_walk.py` qui produit un paysage où il en a, en gardant toutes les dynamiques les plus décomposées puis en y ajoutant leur voisinage :

```
python3 src/python/core_then_walk.py -d <d> [-M <m>] [-n <n>] [-o <fichier>]
      [--min-leaves <k>] [--max-free <k>] [--fanout <k>]
      [--measure variety|genotype] [--seed <s>] [--no-weights]
```

`-M <m>` nombre maximal de dynamiques du cœur décomposé (3000 par défaut) ;

`-n <n>` nombre total de dynamiques, cœur compris (30 000 par défaut) ;

`--min-leaves <k>` nombre minimal de feuilles des arbres du cœur, à augmenter pour un cœur plus petit et plus décomposé (2 si $d \leq 3$, 8 sinon) ;

`--max-free <k>` nombre maximal de nœuds libres par arbre du cœur, à augmenter pour y inclure des dynamiques moins décomposées, au prix du temps de calcul (illimité si $d \leq 3$, 0 sinon) ;

`--fanout <k>` nombre de nouveaux voisins pris par dynamique à chaque étape de l'exploration du voisinage (4 par défaut).

Les autres options sont celles de `mcsbn`, que le script utilise s'il est compilé (et sinon une version Python plus lente).

Les positions dans le métagraphe s'ajoutent à n'importe quel CSV, de MCSBN comme de JPP, avec

```
python3 src/python/add_metagraph_layout.py <entrée> [-o <sortie>] [--vis <Vis>]
      [--neg <k>] [--samples <k>] [--gamma <x>] [--mode <k>] [--A <x>] [--B <x>]
```

où les dernières options sont celles de DRGraph. Par défaut, la sortie s'appelle `<entrée>_metagraph.csv` (ou `.csv.gz`) et `Vis` est cherché dans `tools/DRGraph/`. Le portage en C, `bin/metagraph_layout`, prend les mêmes arguments et va plus vite sur les gros fichiers, mais se limite à $d \leq 4$; son option `--stats-only` n'écrit rien et affiche seulement les composantes connexes du métagraphe.

Enfin, `make test` compare MCSBN aux CSV que JPP produit avec les programmes `treeinsolver` (`<d>d_PSBN_undercontrol_treeinsolver_output.csv` et sa version `PTBN`, dans `JPPipeline/out/`), qu'il faut donc avoir générés. Les autres scripts de `src/python/` sont des modules communs ou des outils intermédiaires.

A.6 Scripts annexes et programmes historiques

Pour ce qui est des autres scripts mis au point, ils se limitent au calcul de la borne des poids et à la génération des fichiers `setup_btree`. La borne des poids B_d et le nombre d'orbites de SBF du tableau 2 se calculent avec

```
python3 SBN_pluripotent/w_bound_orbits.py [--max-n <n>] [--big-m <m>]
```

`--max-n <n>` dimension maximale traitée (5 par défaut), le script étant conçu pour aller jusqu'à 6 ;

`--big-m <m>` borne supérieure des poids dans le programme linéaire en nombres entiers (200 par défaut, largement suffisant).

Le script affiche, pour chaque dimension, la borne, le nombre d'orbites et une SBF qui l'atteint.

Les fichiers `setup_btree_{1..6}d.lp`, déjà fournis dans `JPPipeline/src/asp/`, se régénèrent dans le dossier courant avec

```
python3 SBN_pluripotent/gen_setup_btree.py <d> [<d> ...]
```

Le reste relève des étapes antérieures du stage, à savoir les programmes de Laurent et leurs scripts d'export (`SBN_pluripotent_Laurent/`), l'export `export_btree.py` et le comptage de fréquences par ApproxASP (`JPPipeline/src/asp/freq_counting/`), que remplace le calcul exact de `GenotypeCount` dans JPP.